

COMP3018: Report (Task-3 Literature Review & Task-4 Programming Project)

Cognitive Robotics

Contents

1- Task (3) Literature Review	2
1.1 Introduction	2
1.2 Literature Review	2
1.3. Applications	3
1.3.1 Therapeutic and Emotional Support	3
1.3.2 Medication Adherence and Daily Living Support	3
1.3.3 Physical Rehabilitation and Mobility	4
1.4. Discussion	4
1.4.1 Challenges	4
1.4.2 Ethical Implications	5
1.5 Conclusion	5
1.6. Task-3 References	6
1.6.1 Papers and Journal Articles	6
2- Task (4) Novel Programming Project (Adaptiveness in Assistive Robotics)	8
2.1 Introduction (10%; Salman)	8
2.2 Background (10%; Alfie)	8
2.3. Methods & Setup (35%; Alfie)	9
2.3.1 System Architecture	9
2.3.2 Input Layer: Multimodal and Behavioural Signals	9
2.3.3 Process Layer: Multi-Signal State Inference	13
2.3.4 Generate Layer: Dynamic Prompt Construction	16
2.3.5 Output Layer: Aligned Multimodal Response	16
2.3.6 Adaptation Self-Evaluation and Session Persistence	16
2.4. Outcome & System Analysis (30%; Salman)	17
2.4.1 Overview	17
2.4.2 Local testing	17
2.4.3 Lab session testing	17
2.5 Conclusion (10%; Alfie)	18
2.6. Task-4 References (5%)	18
2.6.1 Academic Books	18
2.6.2 Academic Papers	18
3. Appendix	19
3.1 Video Demo	19
3.2 AI Declaration	20
3.3. Novel Python Project: <code>.env</code> and <code>gaze22.1.py</code>	21
3.3.1 <code>.env</code> :	21

1- Task (3) Literature Review

1.1 Introduction

Ageing populations and a shrinking care workforce positioned **assistive robotics** (*human-supportive robots within physical, cognitive, and social realms of impairment affecting daily-living activities*); a prominent technological response to a widening care gap. The field spans diverse subfields; this essay however focuses on *socially* assistive robotics (*SAR*), an assistive-robot subfield in which the robot “focuses on helping human users through social rather than physical interaction” (Tapus, Matarić and Scassellati, 2007, Abstract), as here most active research and ethical tension converge.

Robots now administer medication reminders, guide rehabilitation exercises, and therapeutically companionate in clinical and domestic settings: reduced caregiver burden, improved patient outcomes; residents became “more active and more communicative, both with each other and their caregivers” (Wada and Shibata, 2007, p. 973).

However, most-current assistive robots operate at what Sciutti et al. (2023, p. 160) call the social layer: they react to immediate stimuli but lack the cognitive depth to anticipate user needs, remember past interactions, or reason about their own performance. Sciutti et al. argue that effective assistive robots should be *cognitive*: capable of “flexible, context-sensitive action, knowing what they are doing and why they are doing it.” Vernon, Metta and Sandini (2007, p. 151) formalise this requirement via a “virtuous cycle that is embedded in an ongoing process of action and perception” (the agent anticipates → learns → adapts to achieve autonomy). This essay contends that assistive robotics should graduate from reactive social behaviour to cognitive capability (intelligence deployed *over* the social layer) for sustained, personalised support.

1.2 Literature Review

Cognitive robotics: defined by Sandini, Sciutti and Vernon (2021, Overview), lies at the intersection of Robotics, Artificial Intelligence, and Cognitive and Biological Sciences, integrating “sensorimotor skills, knowledge representation and reasoning, and social interaction.” This interdisciplinary grounding distinguishes it from conventional robotics (*treats the robot as purely engineered*) and from social robotics (*addresses interaction behaviour without necessarily modelling cognitive processes*). The distinction is consequential: a robot that smiles when a patient smiles is social; a robot that infers *why*, and adapts accordingly, is *cognitive*.

42 catalogued definitions of cognition (euCognition) share a common thread (Vernon, Metta and Sandini, 2007, Abstract): anticipation, learning, and adaptation, intersected with perception and action to create autonomy (Fig. 1). This cycle provides an architectural checklist for assistive robots: a system that cannot direct its gaze toward relevant stimuli and suppress irrelevant ones (*selective attention*), anticipate the outcome of its actions (*i.e. prospection*), learn from past interactions (*memory*), or adapt its strategy when performance declines (*metacognition*) is, per this framework, not yet cognitive. Kotseruba and Tsotsos’s (2020, Abstract) survey of 84 cognitive architectures organises them along these abilities, thus corroborating the checklist’s standing in the field. Sciutti et al. (2023, p. 160) further specify that cognitive robots should “reason about their actions and modify their behavior to improve their effectiveness”; a capacity termed *theory of mind*: the agent infers another’s hidden mental state.

Furthermore, memory is not treated as a single uniform store (monolithic); Laird (2012, Chapter 9) distinguishes *episodic memory* (*records of specific past experiences and their contextual outcomes*) from *semantic memory* (*general knowledge about the world, including spatial relationships and factual constraints*) in implementational terms: episodic memory is “what you ‘remember’”, and semantic memory is “what you ‘know’”. For example, assistive-medication robots need episodic memory to recall that a user refused medication after a restless night, and semantic memory to know certain drugs cannot also be administered.

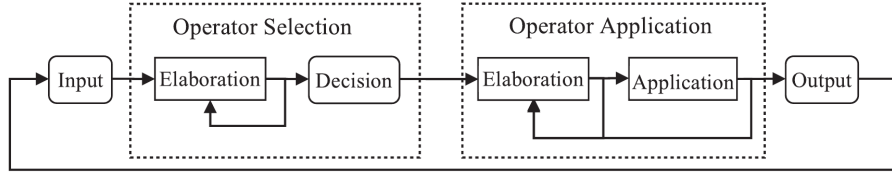


Figure 4.7
The Soar processing cycle.

Figure 1: Soar’s processing cycle, reproduced from Laird (2012, fig. 4.7, p. 79), as a concrete instantiation of Vernon, Metta and Sandini’s (2007) cognition cycle. Laird specifies that the cycle “consists of four phases: Input, Operator Selection, Operator Application, and Output” (Laird, 2012, p. 79), wherein Input maps to perception, Operator Selection draws on episodic and semantic memory to anticipate consequences (i.e. *prospection*), Operator Application executes the chosen behaviour through parallel rule-firing waves, and Output returns the agent to the environment whilst the loop hands control back to Input for continual adaptation. This architecture exposes the deficit of reactive systems: PARO, for instance, implements only Input → Output, lacking the substate-driven Operator Selection where deliberation over latent user states could occur. Furthermore, Laird argues that “the most significant effect of chunking is that it eliminates processing in substates for situations similar to ones experienced in the past” (Laird, 2012, p. 164); repeated deliberations compile into procedural rules; a mechanism distinct from POMDP belief-space approximation, yet complementary thereto in addressing real-time embodied operation under care-time constraints.

1.3. Applications

1.3.1 Therapeutic and Emotional Support

The PARO therapeutic seal robot is among the most-widely deployed platforms within socially assistive robotics (Tapus, Matarić and Scassellati, 2007, .pdf-p. 1). Wada and Shibata (2007, p. 973) demonstrate that PARO improves mood in patients with dementia, utilising tactile and auditory inputs. Clinical trials report that urinary stress indicators “significantly improved” after PARO’s introduction (Wada and Shibata, 2007, p. 978, Table II), therefore the platform has been adopted in care homes.

Notwithstanding these benefits PARO operates at the reactive layer, possessing no theory of mind (cannot infer *why* a patient is agitated (loneliness, pain, confusion) nor episodic memory of *what* calmed the patient previously). A cognitively-equipped therapeutic robot, in contrast, would anticipate mood shifts (*prospection*), recall what calmed the patient (episodic memory), and adapt its strategy (metacognition). PARO’s effectiveness plateaus because it cannot personalise; the gap is therefore clinically important. The architectural cost is: without episodic memory, the agent’s perception is “limited both spatially and temporally—that is, to the here and now” (Laird, 2012, p. 233). Fong, Nourbakhsh and Dautenhahn (2003, p. 145) formalise this gap via Breazeal’s taxonomy (*an ordered category scheme*): PARO occupies the “social interface” level (human-like cues but “shallow models of social cognition”), whereas Sciutti et al.’s (2023, p. 160) vision of robots “knowing what they are doing and why” demands the *socially intelligent* level.

1.3.2 Medication Adherence and Daily Living Support

Medication non-adherence imposes substantial costs on healthcare systems, and elderly patients with polypharmacy regimens particularly are vulnerable to missed or incorrect doses. Robots in this domain face a different challenge from therapeutic companionship: trust and cognitive load are latent variables, inferred only from *noisy-behavioural signals*. Lee and See (2004, .pdf-p. 6) define trust as “the attitude that an agent will help achieve an individual’s goals in a situation characterized by uncertainty and vulnerability”; a definition foregrounding the unobservable nature that necessitates probabilistic modelling. A user might comply with a medication prompt despite low trust (e.g. time pressure), or indeed refuse despite high trust (e.g. task complexity), and thus, the observation alone cannot reliably disambiguate the underlying state (Kaelbling, Littman and Cassandra, 1998, p. 105, 3. *Partial observability*).

The Partially Observable Markov Decision Process (POMDP) provides formal machinery for this uncertainty. Chen et al. (2020, p. 6) demonstrate a Trust-POMDP wherein the robot maintains a belief distribution over trust and selects actions that maximise long-term collaboration, showing belief-space planning outperforms fixed strategies. Garcez and Lamb (2023, .pdf-p. 1) advocate a neuro-symbolic paradigm in which neural subsystems handle perception, and symbolic subsystems (e.g. POMDPs) handle temporal reasoning. Nikolaidis, Hsu and Srinivasa (2017, p. 625) provide empirical corroboration: in a collaborative task ($n = 69$), robots utilising mutual adaptation via a Mixed Observability MDP (modelling human adaptability as a latent variable) were rated significantly more trustworthy than fixed-policy alternatives ($U = 180$, $p = 0.048$). This aligns with Hancock et al.’s (2011, p. 522) finding that performance is the strongest trust predictor.

1.3.3 Physical Rehabilitation and Mobility

Robotic exoskeletons and assistive manipulators for stroke recovery and mobility support need to adapt not only to the patient’s physical state (joint angles and muscle activation patterns) but also to their psychological state: motivation, frustration, and fatigue determine whether a patient perseveres or disengages.

Embodied cognition becomes essential. Brooks (1991, Introduction) argues that intelligence emerges from physical interaction with the environment instead of abstract representation, whereas Fong, Nourbakhsh and Dautenhahn (2003, p. 149) operationalise this as “perturbatory coupling”: the more channels of mutual influence, the more embodied the system. A rehabilitation robot therefore occupies a uniquely cognitive niche, sensing the patient’s body, reasoning about current capabilities, and adapting appropriately. A purely language-based or screen-based interface cannot achieve this; Matarić et al. (2007, .pdf-p. 7) confirm: stroke survivors engaged more with embodied robots than screen-based alternatives. Tapus, Țăpuș and Matarić (2008, Abstract) show that embodiment alone is insufficient: adaptive personality matching (adjusting interaction distance and speed to the user’s traits) further improved task performance.

1.4. Discussion

1.4.1 Challenges

Tapus, Matarić and Scassellati (2007, .pdf-p. 6) projected that by 2012 SAR systems would demonstrate “marked improvement in learning/training/recovery of the user”; yet PARO, the most-deployed platform nearly twenty years later, *still* cannot remember yesterday’s session. Three challenges explain this. Firstly, computational intractability: solving *POMDPs* exactly is PSPACE-complete (Papadimitriou and Tsitsiklis, 1987, p. 448), and the belief simplex (*geometric space of all possible probability distributions over the hidden states*) grows exponentially with state-space dimensionality. Whilst approximate solvers such as point-based value iteration (Pineau, Gordon and Thrun, 2003, p. 1025) and online Monte-Carlo tree search (Silver and Veness, 2010, p. 1) mitigate this, real-time cognitive processing within embodied systems remains an open challenge, particularly when multiple unobserved variables (trust, load, emotion) require tracking simultaneously.

Secondly, the measurement problem: trust, cognitive load, and emotional state are not directly observable; observations thereof are noisy proxies at best. Hancock et al.’s (2011, p. 522) meta-analysis of 29 studies finds that even the strongest correlates of trust explain only moderate variance, but Broadbent, Stafford and MacDonald (2009, Abstract) note that acceptance depends on matching robot behaviour to user expectations, not trust alone. Desai et al. (2013, p. 258, Conclusion) demonstrate trust dynamics are non-linear, building slowly through consistent performance but degrading rapidly after errors; and thus a single misclassified observation can cascade. Nikolaidis, Hsu and Srinivasa (2017, p. 626), however, demonstrate that mutual adaptation partially mitigates this fragility: when the robot models human adaptability as a latent variable, trust persists even during strategy disagreements, suggesting the variance Hancock et al. report may stem from studies that treat the human as static instead of co-adaptive.

Finally, adaptation without exploitation: a robot that runs inference on cognitive load could time its medication requests to coincide with periods of high vulnerability, maximising compliance at the expense of user autonomy. The POMDP reward function should therefore encode ethical constraints alongside clinicians’ objectives.

1.4.2 Ethical Implications

Assistive robots in intimate care spaces (bedrooms, bathrooms, rehabilitation clinics) continuously collect sensitive behavioural data. Facial expressions, vocal patterns, and movement trajectories constitute biometric data, yet regulatory frameworks have not kept pace with deployment. Wachter, Mittelstadt and Floridi (2017, Abstract) argue that even the General Data Protection Regulation provides no enforceable “right to explanation” of automated decisions; a gap particularly concerning in healthcare where recommendations directly affect patient wellbeing.

Moreover, over-reliance on assistive robots risks eroding functional independence. If a robot anticipates and pre-empts needs via prospection, the user may disengage from self-directed activity, contradicting the assistive mandate. Sharkey (2014, *.pdf*-p. 6) frames this via Nordenfelt’s ‘Dignity of Identity’: “a robot that dealt impersonally with an older person, without knowing or using their name or their preferences would also be likely to negatively affect their feelings of dignity.” This implies that only cognitively-equipped robots (with episodic memory of individuals) can avoid dignity violations; reactive systems such as PARO, regardless of their therapeutic benefits (Wada and Shibata, 2007, p. 973), risk infantilisation (*treating the user as an infant*) because they cannot personalise. The responsibility gap compounds this further: when a care robot administers incorrect medication, liability falls ambiguously between manufacturer, deployer, and clinician.

The ethical watchword is therefore proactive regulation: design-stage ethics anticipating failure modes, not reactive patchwork after harm. Per the embodied cognition thesis, if intelligence indeed requires a body, and that body enters the most intimate spaces of vulnerable persons, then the ethical stakes of assistive cognitive robotics are uniquely high.

1.5 Conclusion

Assistive robotics stands at an inflection point. Current systems deliver measurable benefits within narrow envelopes, yet their reactive architectures limit sustained, personalised effectiveness. The Vernon, Metta and Sandini (2007) cognition cycle provides the architectural blueprint for graduating beyond this plateau: assistive robots that anticipate (prospection), remember (episodic and semantic memory), reason about others’ mental states (theory of mind), and monitor their own performance (metacognition) would mean a qualitative advance over current systems.

The neuro-symbolic paradigm offers a viable path toward this vision, as Trust-POMDP attests, with “trust dynamics” learned within symbolic planning (Chen et al., 2020, p. 6). Sciutti et al. (2023, p. 161) independently identify the integration of learning with model-based approaches as cognitive robotics’ most-prominent trajectory; that this converges with Garcez and Lamb’s (2023, *.pdf*-p. 1) *third wave* thesis suggests the direction is not single research group-confined. Sharkey and Sharkey (2012, *.pdf*-p. 1) caution that such systems risk replacing rather than supplementing human-care; the field should therefore pursue cognitive capability and ethical governance in concert, lest the technology displace these very carers it was meant to supplement (Fig.~2). The ultimate test, per the embodied cognition thesis, is a robot that can: sense $\rightarrow$ remember $\rightarrow$ anticipate $\rightarrow$ adapt within the physical world, whilst respecting the autonomy, and dignity, of the persons it serves.

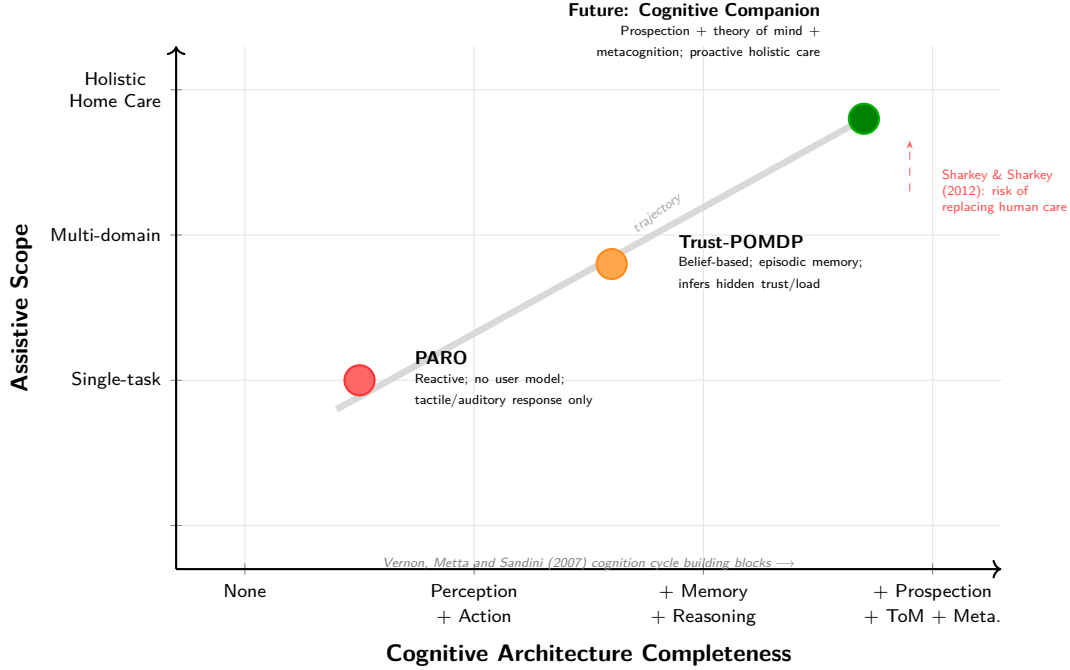


Figure 2: Trajectory of assistive robotics from reactive single-task systems (PARO) through adaptive belief-based architectures (Trust-POMDP) toward cognitively autonomous home-dwelling companions. Expanding assistive scope without expanding cognitive capability is insufficient; the diagonal trajectory requires both.

1.6. Task-3 References

1.6.1 Papers and Journal Articles

- Broadbent, E., Stafford, R. and MacDonald, B. (2009) ‘Acceptance of Healthcare Robots for the Older Population: Review and Future Directions’, *International Journal of Social Robotics*, 1(4), pp. 319-330. Available at: https://www.researchgate.net/publication/220397395_Acceptance_of_Healthcare_Robots_for_the_Older_Population_Review_and_Future_Directions (Accessed: 25 March 2026).
- Brooks, R. A. (1991) ‘Intelligence without representation’, *Artificial Intelligence*, 47(1-3), pp. 139-159. Available at: <https://people.csail.mit.edu/brooks/papers/representation.pdf> (Accessed: 24 March 2026).
- Chen, M., Nikolaidis, S., Soh, H., Hsu, D. and Srinivasa, S. (2020) ‘Trust-Aware Decision Making for Human-Robot Collaboration: Model Learning and Planning’, *ACM Transactions on Human-Robot Interaction*, 9(2), pp. 1-23. Available at: https://personalrobotics.cs.washington.edu/publications/ch_en2019trust.pdf (Accessed: 15 March 2026).
- Desai, M., Kaniarasu, P., Medvedev, M., Steinfeld, A. and Yanco, H. (2013) ‘Impact of robot failures and feedback on real-time trust’, in *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI '13)*, pp. 251-258. Available at: <https://ieeexplore.ieee.org/document/6483596> (Accessed: 20 March 2026).
- Fong, T., Nourbakhsh, I. and Dautenhahn, K. (2003) ‘A survey of socially interactive robots’, *Robotics and Autonomous Systems*, 42(3-4), pp. 143-166. Available at: <https://www.cs.cmu.edu/~illah/PAPERS/socialroboticssurvey.pdf> (Accessed: 18 March 2026).
- Garcez, A. d’A. and Lamb, L. C. (2023) ‘Neurosymbolic AI: The 3rd Wave’, *Artificial Intelligence Review*, 56, pp. 12387-12406. Available at: <https://link.springer.com/article/10.1007/s10462-023-10448-w> (Accessed: 20 March 2026).
- Hancock, P. A., Billings, D. R., Schaefer, K. E., Chen, J. Y. C., de Visser, E. J. and Parasuraman, R. (2011) ‘A meta-analysis of factors affecting trust in human-robot interaction’, *Human Factors*, 53(5), pp. 517-527. Available at: <https://journals.sagepub.com/doi/10.1177/0018720811417254> (Accessed: 20 March 2026).

15 March 2026).

- Kaelbling, L. P., Littman, M. L. and Cassandra, A. R. (1998) ‘Planning and acting in partially observable stochastic domains’, *Artificial Intelligence*, 101(1-2), pp. 99-134. Available at: <https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf> (Accessed: 13 March 2026).
- Kotseruba, I. and Tsotsos, J. K. (2020) ‘40 years of cognitive architectures: core cognitive abilities and practical applications’, *Artificial Intelligence Review*, 53(1), pp. 17-94. Available at: <https://link.springer.com/article/10.1007/s10462-018-9646-y> (Accessed: 3 May 2026).
- Laird, J. E. (2012) *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press.
- Lee, J. D. and See, K. A. (2004) ‘Trust in automation: Designing for appropriate reliance’, *Human Factors*, 46(1), pp. 50-80. Available at: https://journals.sagepub.com/doi/10.1518/hfes.46.1.50_30392 (Accessed: 15 March 2026).
- Matarić, M. J., Eriksson, J., Feil-Seifer, D. J. and Winstein, C. J. (2007) ‘Socially assistive robotics for post-stroke rehabilitation’, *Journal of NeuroEngineering and Rehabilitation*, 4(5), pp. 1-9. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC1821334/> (Accessed: 25 March 2026).
- Nikolaidis, S., Hsu, D. and Srinivasa, S. (2017) ‘Human-robot mutual adaptation in collaborative tasks: Models and experiments’, *The International Journal of Robotics Research*, 36(5-7), pp. 618-634. Available at: <https://journals.sagepub.com/doi/10.1177/0278364917690593> (Accessed: 20 March 2026).
- Papadimitriou, C. H. and Tsitsiklis, J. N. (1987) ‘The complexity of Markov decision processes’, *Mathematics of Operations Research*, 12(3), pp. 441-450. Available at: <https://web.mit.edu/jnt/www/Papers/J016-87-mdp-complexity.pdf> (Accessed: 13 March 2026).
- Pineau, J., Gordon, G. and Thrun, S. (2003) ‘Point-based value iteration: An anytime algorithm for POMDPs’, in *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI-03)*, pp. 1025-1030. Available at: <http://www.cs.cmu.edu/~ggordon/jpineau-ggordon-thrun.ijcai03.pdf> (Accessed: 24 March 2026).
- Sandini, G., Sciutti, A. and Vernon, D. (2021) ‘Cognitive Robotics’, in *Encyclopedia of Robotics*. Berlin: Springer-Verlag. Available at: http://www.vernon.eu/publications/2021_Sandini_et_al.pdf (Accessed: 3 May 2026).
- Sciutti, A., Beetz, M., Inamura, T., Korsah, A., Oh, J., Sandini, G., Shimoda, S. and Vernon, D. (2023) ‘The Present and the Future of Cognitive Robotics’, *IEEE Robotics & Automation Magazine*, 30(3), pp. 160-163. Available at: <https://ieeexplore-ieee-org.plymouth.idm.oclc.org/document/10255092> (Accessed: 18 March 2026).
- Sharkey, A. (2014) ‘Robots and human dignity: A consideration of the effects of robot care on the dignity of older people’, *Ethics and Information Technology*, 16(1), pp. 63-75. Available at: <https://philarchive.org/rec/SHARAH-2> (Accessed: 22 March 2026).
- Sharkey, A. and Sharkey, N. (2012) ‘Granny and the robots: ethical issues in robot care for the elderly’, *Ethics and Information Technology*, 14(1), pp. 27-40. Available at: <https://philarchive.org/rec/SHAGAT> (Accessed: 22 March 2026).
- Silver, D. and Veness, J. (2010) ‘Monte-Carlo planning in large POMDPs’, in *Advances in Neural Information Processing Systems (NeurIPS 23)*, pp. 2164-2172. Available at: <https://proceedings.neurips.cc/paper/2010/file/edfbc1afcf9246bb0d40eb4d8027d90f-Paper.pdf> (Accessed: 24 March 2026).
- Tapus, A., Matarić, M. J. and Scassellati, B. (2007) ‘Socially assistive robotics [Grand Challenges of Robotics]’, *IEEE Robotics & Automation Magazine*, 14(1), pp. 35-42. Available at: <https://sczlab.yale.edu/sites/default/files/files/Tapus-RAM2007.pdf> (Accessed: 25 March 2026).
- Tapus, A., Țăpuș, C. and Matarić, M. J. (2008) ‘User-robot personality matching and assistive robot behavior adaptation for post-stroke rehabilitation therapy’, *Intelligent Service Robotics*, 1(2), pp. 169-183. Available at: <https://hal.science/hal-00770108/document> (Accessed: 26 March 2026).
- Vernon, D., Metta, G. and Sandini, G. (2007) ‘A Survey of Artificial Cognitive Systems: Implications for the Autonomous Development of Mental Capabilities in Computational Agents’, *IEEE Transactions on Evolutionary Computation*, 11(2), pp. 151-180. Available at: https://www.robotcub.org/misc/papers/07_Vernon_Metta_Sandini_IEEE.pdf (Accessed: 13 March 2026).
- Wachter, S., Mittelstadt, B. and Floridi, L. (2017) ‘Why a Right to Explanation of Automated Decision-Making Does Not Exist in the General Data Protection Regulation’, *International Data Privacy Law*, 7(2), pp. 76-99. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2903469

(Accessed: 22 March 2026).

- Wada, K. and Shibata, T. (2007) ‘Living with seal robots: its sociopsychological and physiological influences on the elderly at a care house’, *IEEE Transactions on Robotics*, 23(5), pp. 972-980. Available at: <https://ieeexplore.ieee.org/document/4339551> (Accessed: 18 March 2026).

2- Task (4) Novel Programming Project (Adaptiveness in Assistive Robotics)

2.1 Introduction (10%; Salman)

Gaze is an adaptive game hosting engagement coaching system that is designed to run on the pepper robot. it turns pepper into a responsive companion that plays both mathematical and letter countdown games while observing and adapting to the human. Gaze is a multi-signal emotional inference engine as the system cross validates three live data streams: facial expression recognition for emotional states, speech expression recognition and an engagement tracking feature that monitors response time, silence time and answer accuracy. Using these features helps recognising real emotional states and reduce risk or misleading readings for example a neutral face with slow responses and low accuracy means disengagement however tense expressions with fast and correct responses means focus.

Inspired by assistive robots gaze role is to keep users engaged. To ensure that it’s doing its job gaze detects when attention is fading and steps in to reengage by changing difficulty level and offering hints if the user is struggling. The questions are generated dynamically using Open AI which also verifies answers and shapes the answers based on users inferred state. The speech is transcribed using whisper which allows the system to freely form answers with responses combine speech, gestures and LED feedback. All progress is saved making gaze a system that doesn’t just host games but also adapts and coaches

2.2 Background (10%; Alfie)

GAZE sits within socially assistive robotics (*the deployment of robots to support users through social interaction instead of physical contact*) and affective computing, which Picard (1997, p. 1, Abstract) founds as “computing that relates to, arises from, or influences emotions”; Tapus, Matarić and Scassellati (2007, .pdf-p. 1) define this sub-field as systems that “assist users through social interaction.” Most-current platforms react to a single input signal, suffering the single-signal problem: a facial-expression classifier misreads resting faces as displeasure; a response-time metric mistakes deliberation for disengagement. Poria et al. (2017, p. 99) report multimodal systems were “consistently (85% of systems) more accurate than their best unimodal counterparts, with an average improvement of 9.83%”; Spezialetti, Placidi and Rossi (2020, pp. 1-2, Introduction) substantiate this within HRI via reviewings of recognition systems across facial, vocal, and physiological channels. This demands multi-signal fusion (the system weighs complementary channels together instead of trusting any one alone).

GAZE’s core contribution: multi-signal emotional inference, fusing six channels with derived temporal signals into a single-inferred user-state. The implementation is face-primary: voice nudges only when the face is Neutral and the MLP clears 0.9 confidence (**fearful** excluded as the silence-attractor). **gpt-5.4** generates dialogue; the symbolic AdaptiveEngine governs state inference, grounding output in Pepper’s affordances (Ahn et al., 2022, .pdf-p. 1, Abstract); a **gpt-4.1** verifier handles answer-checking. Smedegaard (2019, p. 414, Experiential novelty) reviews the conventional assumption that engagement reflects novelty instead of sustained interest, before reframing novelty effects themselves as informative signals about user sense-making. GAZE’s adaptive engine therefore aims to sustain engagement beyond the novelty phase by adaptively (dynamically) adjusting difficulty, switching games, and drafting adapted questions based on inferred user-state.

2.3. Methods & Setup (35%; Alfie)

2.3.1 System Architecture

GAZE’s per-turn cycle (Fig.~3) is neuro-symbolic (*LLM does the talking, hand-coded rules do the deciding*): `gpt-5.4` controls dialogue and decision-making whilst the AdaptiveEngine and game logic serve as callable tools, aligning with Garcez and Lamb’s (2023, .pdf-p. 1) *third wave* paradigm wherein neural and symbolic share a structured interface (cf. Ahn et al., 2022, .pdf-p. 1).

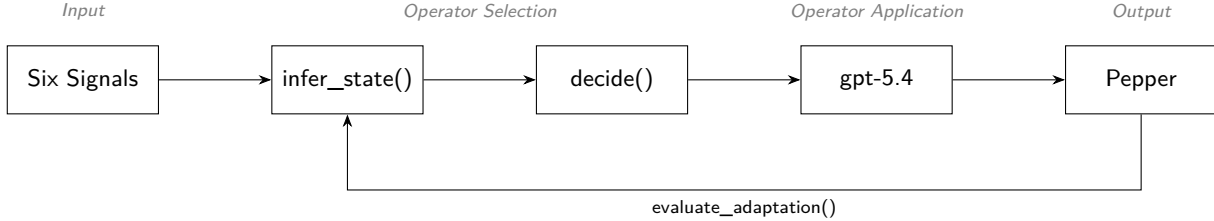


Figure 3: GAZE’s per-turn cycle mapped onto Laird’s (2012, fig. 4.7, p. 79) Soar phases. Each turn, six live signals fan into the AdaptiveEngine, ‘gpt-5.4’ generates an adapted response via function calling, and Pepper executes speech, gesture, and LED concurrently: six signals feed Input, `infer_state()` and `decide()` perform Operator Selection, `gpt-5.4` handles Operator Application; *Pepper* executes Output. Unlike Soar’s implicit chunking, GAZE closes the loop via an explicit `evaluate_adaptation()` step that carries to the next turn, the loop is conversational, not question-answer.

2.3.2 Input Layer: Multimodal and Behavioural Signals

1- Facial Expression (vision-based). A pre-trained CNN (Workshop 10) classifies the user’s expression into six Ekman categories plus a Neutral (*‘don’t know’ thus classify as expressionless*) class, building upon Ekman and Friesen (1971, p. 128), whose cross-cultural results show “particular facial behaviors are universally associated with particular emotions”; this taxonomy remains “still the most popular perspective for FER” (Li and Deng, 2020, p. 1). Known limitations (cultural bias, resting-face misclassification) are mitigated by combining facial with behavioural signals (§2.3.3).

The pipeline is two-stage: a Haar cascade (*OpenCV’s Viola-Jones detector; locates faces via fast rectangular brightness-contrast features*) first finds the face’s bounding box on a 2×-downscaled greyscale frame, then the CNN classifies emotion within the largest face ROI. Two daemon threads decouple capture from inference at \$ \$6.7 Hz (turn-based interaction does not warrant 30 Hz); on Pepper, frames stream over TCP via persistent `ALVideoDevice` instead of per-turn SFTP.

2- Verbal Answer (speech-based). Pepper’s four-microphone polling (Listing~1) catches off-axis speakers a front-only poll would miss; `force_mono_16k_wav()` canonicalises (*forces one shape on every clip*) to mono 16 kHz. Whisper (`whisper-1`), whose models “approach [human] accuracy and robustness” (Radford et al., 2023, Abstract), serves as a delivery-aware sensor: `verbose_json` exposes `no_speech_prob`, `avg_logprob`, and `compression_ratio`, encoding *how* the user spoke alongside *what* they said. Barge-in (*letting the user interrupt Pepper mid-utterance*) was omitted: NAOqi’s imperfect echo cancellation would fire false interrupts.

Listing 1: `nao_record`: calibrated-threshold silence detection polling the max energy across all four Pepper mics, with firmware fallback. Recording uses the [1,1,1,1] channel mask hence catching off-axis speakers a front-only poll would miss; recording terminates after 1.2 s of post-speech silence or at the adaptive hard ceiling. `force_mono_16k_wav()` downstream picks the loudest channel by RMS so Whisper, Silero-VAD, the WS-08 MLP, and `measure_volume()` all see one canonical mono 16 kHz layout.

```

1 def nao_record(ssh, energy_threshold: int = DEFAULT_ENERGY_THRESHOLD,
2               record_max_secs: float = RECORD_MAX_SECS,
3               silence_secs: float = SILENCE_DURATION):
4     """Record audio on Pepper with dynamic silence detection.
5     - poll the max energy across all four mics to stop recording early if silence detected

```

```

6         - calibrated energy threshold to avoid false positives from ambient noise
7         - if getFrontMicEnergy is unsupported (e.g. older firmware), fall back to a safe
          ↪ fixed-duration recording to ensure the demo still works, albeit without silence
          ↪ detection
8     """
9
10    nao_run(ssh, f"""
11    from naoqi import ALProxy
12    import time
13
14    rec = ALProxy("ALAudioRecorder", "127.0.0.1", 9559)
15
16    rec.stopMicrophonesRecording()
17    rec.startMicrophonesRecording("{REMOTE_WAV}", "wav", 16000, [1, 1, 1, 1])
18
19    try:
20        audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
21
22        speech_detected = False
23        silence_start = None
24        start = time.time()
25        threshold = {energy_threshold}
26
27        while True:
28            elapsed = time.time() - start
29
30            # hard ceiling; never exceed max duration
31            if elapsed >= {record_max_secs}:
32                break
33
34            # poll all four mics and take the MAX; a user stood to either side of Pepper registers on the
          ↪ side mics but not the front, so front-only polling misses them and the silence-detector
          ↪ stops recording mid-utterance
35            try:
36                energy = max(
37                    audio.getFrontMicEnergy(),
38                    audio.getLeftMicEnergy(),
39                    audio.getRightMicEnergy(),
40                    audio.getRearMicEnergy(),
41                )
42            except Exception:
43                energy = audio.getFrontMicEnergy() # firmware fallback
44
45            if elapsed < {RECORD_MIN_SECS}:
46                # minimum recording period
47                if energy > threshold:
48                    speech_detected = True
49                    time.sleep({SILENCE_POLL_SECS})
50                    continue
51
52            if energy > threshold:
53                speech_detected = True
54                silence_start = None
55            else:
56                if speech_detected and silence_start is None:
57                    silence_start = time.time()
58                if speech_detected and silence_start is not None:
59                    if (time.time() - silence_start) >= {silence_secs}:
60                        break

```

```

61
62     time.sleep({SILENCE_POLL_SECS})
63
64 except Exception as e:
65     # firmware fallback; getFrontMicEnergy() unsupported on this Pepper
66     # fall back to a safe fixed-duration recording so the demo never breaks
67     print(" [Silence detection failed: " + str(e) + "] Falling back to fixed-duration recording")
68     time.sleep({record_max_secs})
69
70 rec.stopMicrophonesRecording()
71 """
72     sftp = ssh.open_sftp()
73     sftp.get(REMOTE_WAV, LOCAL_WAV)
74     sftp.close()

```

Transcription is gated by a five-layer defence chain (Listing~2).

Listing 2: `transcribe`: five-layer defence chain against Whisper hallucination on near-silent audio (`gaze.py`, lines 1532–1610). Whisper emits training-set tics on near-silent audio (CJK gibberish, prompt-primed repetition); the latter is NLG *degeneration* where models get “stuck in repetitive loops” (Ji et al., 2023, p. 3). The chain gates: Silero VAD → Vosk wake-word → Whisper API → `no_speech_prob/avg_logprob/compression_ratio` self-signal diagnostics → normalised hallucination blacklist.

```

1 def transcribe(bypass_wake_word: bool = False,
2               whisper_prompt: str = "User answers a quiz or chats with a companion robot.",
3               record_again=None,
4               max_hallucination_retries=None) -> str:
5     # Silero VAD -> wake-word -> Whisper -> hallucination blacklist
6     attempts_remaining = max_hallucination_retries
7     while True:
8         if INPUT_IS_LOCAL and not _local_speech_detected:
9             return ""
10
11     # Silero VAD hard gate; NAO + LOCAL
12     if not has_real_speech(LOCAL_WAV):
13         print(" Silero VAD found no speech; skipping Whisper.")
14         return ""
15
16     # Vosk wake-word gate; bypass for name prompt
17     if not bypass_wake_word and not has_wake_word(LOCAL_WAV):
18         print(" No wake-word detected; skipping Whisper.")
19         return ""
20
21     try:
22         with open(LOCAL_WAV, "rb") as fh:
23             resp = client.audio.transcriptions.create(
24                 model="whisper-1", #
25                 file=fh, #
26                 response_format="verbose_json", # get self-signals for hallucination detection
27                 temperature=0.0, # zero randomness
28                 prompt=whisper_prompt,
29                 timeout=API_TIMEOUT,
30             )
31             text = (getattr(resp, "text", "") or "").strip()
32             print(f" Whisper raw text: {text!r}")
33
34     # Whisper self-signals from verbose_json
35     segments = getattr(resp, "segments", None) or []

```

```

36     suspected_hallucination = False
37     if segments:
38         no_speech_vals = [s.no_speech_prob for s in segments
39                             if getattr(s, "no_speech_prob", None) is not None]
40         logprob_vals = [s.avg_logprob for s in segments
41                         if getattr(s, "avg_logprob", None) is not None]
42         compression_vals = [s.compression_ratio for s in segments
43                             if getattr(s, "compression_ratio", None) is not None]
44         print(f" Whisper diagnostics: no_speech={no_speech_vals},
45               ↪ avg_logprob={logprob_vals}, compression={compression_vals}")
46         if no_speech_vals and max(no_speech_vals) > 0.6:
47             print(f" Whisper flagged silence (max no_speech_prob={max(no_speech_vals):.2f});
48                   ↪ dropping {text!r}")
49             return ""
50         if logprob_vals and min(logprob_vals) < -1.3:
51             print(f" Whisper low-confidence (min avg_logprob={min(logprob_vals):.2f});
52                   ↪ dropping {text!r}")
53             return ""
54         # repetition-loop hallucination; flag for retry path
55         if compression_vals and max(compression_vals) > 2.4:
56             print(f" Whisper repetition loop (max
57                   ↪ compression_ratio={max(compression_vals):.2f}); flagging {text!r} as
58                   ↪ hallucination")
59             suspected_hallucination = True
60
61     # normalised hallucination blacklist + Whisper-self-signal hallucinations
62     if suspected_hallucination or is_known_hallucination(text):
63         print(f" Filtered Whisper hallucination: {text!r}")
64         if record_again is not None and (attempts_remaining is None or attempts_remaining >
65             ↪ 0):
66             if attempts_remaining is not None: # whilst more retries remain
67                 attempts_remaining -= 1
68                 print(f" Disregarding hallucination; listening again (attempts left:
69                       ↪ {attempts_remaining}).")
70             else:
71                 print(f" Disregarding hallucination; listening again.")
72                 record_again()
73                 continue
74     return ""
75
76     # Strip leading "Pepper"/"Gaze" so handlers receive just the answer; \b blocks
77     ↪ "Pepperoni"/"Gazebo" false positives..
78     stripped = re.sub(r'(?i)^\s*(pepper|gaze)\b[.,\s]*', '', text).strip()
79     return stripped
80 except Exception as e:
81     print(f" Whisper transcribe failed ({e}); returning empty")
82     return ""

```

3- Vocal Emotion (audio-based). The same WAV passes through a pre-trained MLP (Workshop 8) *before* transcription, classifying vocal state via MFCC + chroma + mel-spectrogram features (Listing-3); fear’s tremor however collapses indistinguishable from low-energy real-room speech, hence voice is engineered as a tie-breaker (§2.3.3), an architectural response to a documented limitation (El Ayadi, Kamel and Karray, 2011, p. 573, “acted emotions tend to be more exaggerated than real ones”) rather than a debugging afterthought.

Listing 3: `SpeechEmotionModel.extract_features`: MFCC + chroma + mel-spectrogram feature vector matching the WS-08 training pipeline, with peak-normalisation for quieter brain-injured users (gaze.py, lines 264–292). The MLP classifies vocal state into four emotions (calm, happy, fearful, disgust); El Ayadi, Kamel and Karray (2011, p. 578) identify MFCCs as “the most promising features” for speech-emotion recognition, supplying a second independent modality. RAVDESS is acted-speech (Williams and Stevens, cited in El Ayadi et al., 2011, p. 573, “acted emotions tend to be more exaggerated than real ones”); WS-08 mean-pooling loses “temporal information present in speech signals” (El Ayadi, Kamel and Karray, 2011, p. 574).

```

1
2 @staticmethod # static because it's also used independently in classify_speech_emotion()
3     def extract_features(wav_path: str):
4         "Extract the same MFCC/chroma/mel feature vector used in WS-08 training."
5         with sf.SoundFile(wav_path) as sound_file:
6             audio = sound_file.read(dtype="float32")
7             sample_rate = sound_file.samplerate
8
9         if audio.ndim > 1:
10             audio = audio.mean(axis=1)
11
12         # peak-normalise; helps quieter speakers
13         audio = librosa.util.normalize(audio)
14
15         n_fft = 2048
16         if len(audio) < n_fft:
17             return None
18
19         stft = np.abs(librosa.stft(audio, n_fft=n_fft))
20
21         mfccs = np.mean(librosa.feature.mfcc(
22             y=audio, sr=sample_rate, n_mfcc=40).T, axis=0).flatten()
23
24         chroma = np.mean(librosa.feature.chroma_stft(
25             S=stft, sr=sample_rate).T, axis=0).flatten()
26
27         mel = np.mean(librosa.feature.melspectrogram(
28             y=audio, sr=sample_rate).T, axis=0).flatten()
29
30         return np.concatenate([mfccs, chroma, mel])

```

4- Response Time (engagement-based). A Python timer measures elapsed time from question delivery to recording completion; the Whisper call occurs *after* the timer halts, isolating deliberation time from API latency.

5- Answer Correctness (task-based). gpt-4.1’s `check_game_answer` evaluates the transcribed answer at a deterministic temperature; the resultant binary correctness feeds the AdaptiveEngine and the derived rolling-correctness signal.

6- Volume RMS (arousal-based). `measure_volume()` returns the audio’s RMS (*root mean square; one loudness number per chunk, computed by squaring each sample, averaging, then square-rooting*) over the mono 16 kHz WAV; LOCAL mode auto-calibrates against ambient noise whilst Pepper uses static defaults, hence volume cross-checks face evidence as a tie-breaker.

2.3.3 Process Layer: Multi-Signal State Inference

The AdaptiveEngine’s `infer_state()` classifies the user into one of five states (*Thriving, Comfortable, Struggling, Frustrated, Disengaged*) from six raw inputs supplemented by three derived temporal features (*rolling correctness over the last five rounds, consecutive-silence count, consecutive-wrong streak length*); classification is face-primary (Fig.~4), addressing the instability Desai et al. (2013, Abstract) observe in single-signal systems. Thresholds were pilot-derived.

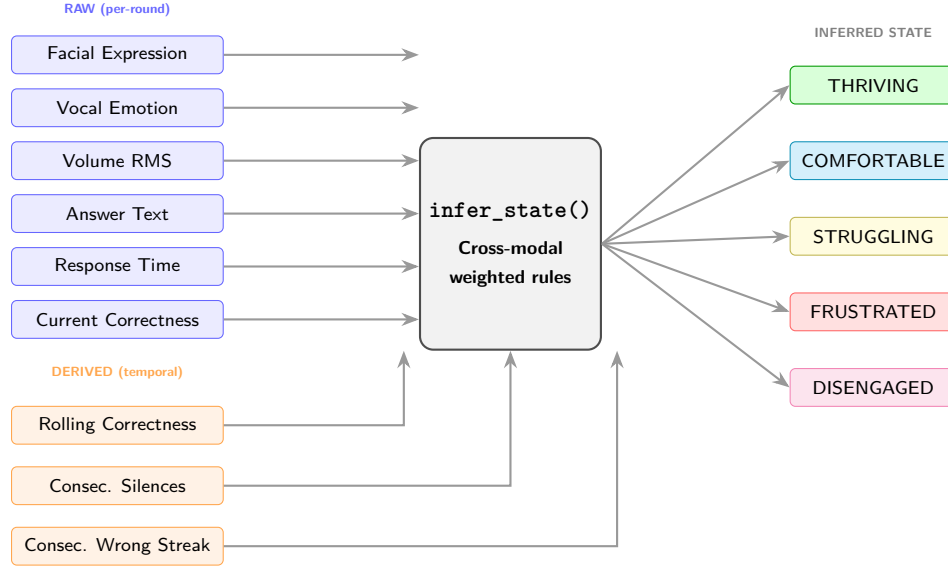


Figure 4: Multi-signal inference pipeline. Six raw inputs captured each round and three derived temporal signals computed from session history feed into `infer_state()`, which applies cross-modal rules (e.g. camera reads *Angry* but user answers fast and correctly $\rightarrow$ *Comfortable*) to classify the user into one of five states. Voice is treated as advisory instead of dominant: it nudges the state only when the face is Neutral AND the MLP’s confidence clears 0.9 AND the label is not *fearful* (the silence-attractor), thus a noisy vocal label can never override stronger visual or behavioural evidence; Busso et al. (2004)’s results indeed report 85.1% unimodal accuracy for facial expression against 70.9% for speech; therein resides the multi-signal novelty.

Listing 4: Multi-signal inference rules (verbatim excerpt from `gaze.py infer_state()`; method-body indentation stripped). The newest implementation is face-primary: every rule fires off facial expression + behavioural signals first, voice is consulted only as a high-confidence tie-breaker at the end. Local-mode arousal calibration: `VOLUME_QUIET = max(ambient  $\times$  2, 200)`, `VOLUME_LOUD = max(ambient  $\times$  10, 2000)`; Pepper-mode applies static defaults (200/2000).

```

1
2 def infer_state(self, expression: str, response_time: float,
3                 correct: bool, answer_text: str,
4                 vocal_emotion: str = "neutral",
5                 vocal_conf: float = 0.0,
6                 volume_rms: float = 0.0) -> InferredState:
7     """
8     Infer the user's state from all signals.
9     Face is primary; voice is only derived when face is Neutral and
10    the voice signal is high-confidence and not "fearful" (the MLP
11    collapses to "fearful" in silence).
12    """
13    correctness = self.rolling_correctness()
14    clean = answer_text.strip().lower()
15    is_silent = (not clean or clean in {"i don't know", "skip", "pass", "next"}) # if no
         $\hookrightarrow$  meaningful input || input matches a skip-command phrase in the set (set over list
         $\hookrightarrow$  because it's faster) then indeed silent (True)
16
17    # Arousal bounds calibrated against ambient noise
18    high_arousal = volume_rms > self.VOLUME_LOUD
19    low_arousal = 0 < volume_rms < self.VOLUME_QUIET
20
21    # voice trusted only when not-fearful

```

```

22         trust_voice = (vocal_conf >= 0.9 and vocal_emotion != "fearful")
23
24     if is_silent:
25         self.consecutive_silences += 1
26     else:
27         self.consecutive_silences = 0
28     if correct:
29         self.consecutive_correct += 1
30         self.consecutive_wrong = 0
31     else:
32         self.consecutive_wrong += 1
33         self.consecutive_correct = 0
34
35     # 1- FACE-PRIMARY RULES: these fire before voice is ever consulted
36
37     # thriving: good performance + fast responses
38     if (correctness >= CORRECTNESS_CEILING and response_time < RESPONSE_TIME_BASELINE * 0.5):
39         return InferredState.THRIVING
40     if expression == "Angry" and correct and response_time < RESPONSE_TIME_BASELINE * 0.6:
41         return InferredState.COMFORTABLE
42
43     # disengaged: silence + slow + poor performance
44     if self.consecutive_silences >= SILENCE_THRESHOLD:
45         return InferredState.DISENGAGED
46     if (expression == "Neutral"
47         and response_time > RESPONSE_TIME_BASELINE
48         and correctness < 0.5):
49         return InferredState.DISENGAGED
50     if (low_arousal and expression == "Neutral"
51         and response_time > RESPONSE_TIME_BASELINE * 0.8):
52         return InferredState.DISENGAGED
53
54     # frustrated: negative face + poor performance
55     if expression in ("Angry", "Disgust") and correctness < CORRECTNESS_FLOOR:
56         return InferredState.FRUSTRATED
57     if self.consecutive_wrong >= 3 and expression in ("Angry", "Sad", "Fear"):
58         return InferredState.FRUSTRATED
59     if (high_arousal and expression in ("Angry", "Disgust", "Fear")
60         and correctness < CORRECTNESS_FLOOR):
61         return InferredState.FRUSTRATED
62
63     # struggling: sadness + slow; poor correctness; fear + wrong
64     if expression == "Sad" and response_time > RESPONSE_TIME_BASELINE * 0.7:
65         return InferredState.STRUGGLING
66     if correctness < CORRECTNESS_FLOOR:
67         return InferredState.STRUGGLING
68     if expression == "Fear" and not correct:
69         return InferredState.STRUGGLING
70
71     # 2- voice tie-breakers; only when face neutral
72     if expression == "Neutral" and trust_voice:
73         if vocal_emotion == "happy" and correct and correctness >= CORRECTNESS_CEILING:
74             return InferredState.THRIVING
75         if vocal_emotion == "calm" and correctness >= 0.5:
76             return InferredState.COMFORTABLE
77
78     return InferredState.COMFORTABLE # default: face gave no negative signal, performance is
       ↪ holding

```

The `decide()` function maps the inferred state to concrete adaptive actions, paralleling Nikolaidis, Hsu and Srinivasa’s (2017, p. 625) mutual-adaptation paradigm; Table~1 summarises the mapping.

Inferred State	Difficulty	Game Switch	Hint	Tone	LED Colour
Thriving	↑ ramp up	No	No	Energetic	Green
Comfortable	↑ if >70%	No	No	Neutral	Cyan
Struggling	↓ ease off	No	Yes	Encouraging	Yellow
Frustrated	↓↓ Easy	After 3 wrong	No	Calm	Red
Disengaged	—	After 3 checkouts	No	Energetic	Magenta

Table 1: State-to-action mapping. The adaptive engine translates each inferred state into a specific combination of difficulty adjustment, game-type switch, hint provision, tone selection, and LED colour. Arrows indicate direction of difficulty change relative to the current level; the mapping applies solely to `decide()`’s engine output (the spoken-checkout path, e.g. repeated *skip*). Disengagement is flagged by three OR’d rules: 1) two consecutive silences (mic-empty or *skip/pass* checkout phrases), 2) neutral-face + response > 15 s + rolling-correctness < 50% (cognitive checkout), 3) low-volume + neutral-face + slow response (early-warning drift); the main-loop then tiers its intervention, tier-1 check-in at 3 silences and tier-2 switch at 4 each firing once per silent spell, bypassing `decide()` entirely. The graduated escalation aligns with Desai et al.’s (2013, p. 256) finding that, post-reliability-drop, ”the rate of recovery is lower than it would otherwise be”; a gentle check-in therefore precedes any disruptive game-switch.

Disengagement is flagged by three OR’d rules; GAZE tiers its intervention (Table~1).

Complementing state inference, `recommend_think_budget()` sets per-turn timing thresholds (*no-speech max*, *silence tolerance*, *recording ceiling*) by combining five signals; the LLM’s `waiting` flag is one among many, not the sole path. `request_more_time` extends the budget *additively*, the four behavioural signals extend independently.

2.3.4 Generate Layer: Dynamic Prompt Construction

GAZE utilises OpenAI function calling to let `gpt-5.4` decide actions via four callable tools; the LLM calls none during natural chat, sequences them during gameplay. Game-question generation runs as a separate JSON-only call with a do-not-repeat memory, preventing mode-collapsed targets (*repeated narrow outputs that ignore prompt variation*). A deterministic `gpt-4.1` verifier (temperature 0.0) handles answer verification, reducing the hallucination risk Ji et al. (2023, p. 3) identify.

A second safeguard inside `build_signal_context()` *semantically buckets* signals (e.g. `System pacing: relaxed` ⇨ `and patient` instead of raw `Think budget: 18s`), preventing the temperature (0.8) from echoing integers verbatim; a failure mode Ji et al. (2023, p. 3) classify under NLG hallucination. After `check_game_answer` fires, same-chain `generate_game_question` is deferred behind: `engine.decide()` updates difficulty first thus preserving adaptation across the neuro-symbolic boundary.

2.3.5 Output Layer: Aligned Multimodal Response

Speech and LED fire concurrently via threading; correct answers additionally fire a celebratory gesture. LED colours reflect the inferred state (Table~1); EarLeds turn blue during LLM inference. Before TTS, `clean_for_tts()` strips markup and applies NFKC normalisation; `split_into_sentences()` chunks at sentence boundaries with 0.4 s pauses for natural delivery. A dedicated TTS-only SSH connection isolates speech from camera + gesture traffic.

2.3.6 Adaptation Self-Evaluation and Session Persistence

`evaluate_adaptation()` compares this round’s with the prior’s to judge whether a previous adaptation helped; session progress saves to `gaze_save.json` every round, with a belt-and-braces auto-save every two turns in

case of interruption.

2.4. Outcome & System Analysis (30%; Salman)

2.4.1 Overview

Gaze was evaluated through local mode on laptop and using the Nao robot in labs. Each session involving a participant seated opposite pepper robot engaging in a game and testing each feature and the outcome. The local mode allowed continuous iteration and was replacing the robots camera and microphone with the laptops while keeping everything else the same.

2.4.2 Local testing

The majority of the testing was done through local mode allowing to test the system calibration, recording, openAI question generation, question validation, facial expression detection and session save and resume to be tested without needing the robot. all the issues were detected and solved during the local testing like whisper hallucinations and question generation and validation as after first testing the system was showing correct to false answers so a validation function was added. After iteration testing the system repeated the same questions and to fix it `gpt-5.4` was used to ban questions that were used forcing it to create new questions and save each session instead of every 5 as some people won't play 5 questions and to make sure their progress is saved all time it was changed to save every round. One of the main set backs was hallucinations as when the mic captures ambient noise whisper would return youtube phrases from the training data rather than emptying the string this was fixed using Solero VAD as a check before whisper and banned words list for the common hallucinations however with short words like yes or fresh were sometimes dropped as logprob scores were below threshold to balance it the threshold was turned down from -0.85 to -1.3 after iteration testing making it rejecting hallucination from short answers.

2.4.3 Lab session testing

The testing with the robot was conducted through 6 lab sessions with the Nao robot which had hardware that the system wasn't tested on yet. The ambient noise calibration returned level of zero energy as the get front mic energy wasn't supported on the Nao robot means that silence detection never triggered and robot records the full duration around 15 seconds each rather than stopping after speech detected which caused longer responses compared to local mode. The recording max was reduced from 8 to 6 seconds to help the long replies. A hybrid mode was also introduced using external camera and microphone from the laptop for better facial expression detection and speech detection while still interacting with the robot. this solved the silence detection issue as the laptop microphone supports the energy calibration so the system can detect when the user has stopped speaking now and stop recording after it detects 2 seconds of silence rather than waiting the whole speech max duration. The hybrid mode also improved facial expression detection as the laptop camera shows a more clear image of the users face compared to the Nao robot as its important to establish the state as sit relies on facial expressions and this change resulted in better state detection giving more accurate results. During the robot testing it was noted that the robot gesture movement wasn't smooth and was too fast which was a robot safety concern as could lead to robots arm it getting stuck or hitting its body while performing a gesture. The interpolation time for all joint movements were increased from 1 second to 2-3 seconds and a safe park as a checkpoint before any movement is made its set into the safe position then goes away from body to ensure it doesn't hit then arm goes up to perform the gesture. This change made the gesture movements safer and smoother to meet safety standards. it was noted that gestures were firing too much slowing the interaction down and having more attention on gestures rather than the game. The gesture logic was reviewed then reduced so that the robot only waves at the start of the session to say hello as a welcoming and when a user gets a correct answer as a celebration he waves twice to make it different from a hello wave and at the end of a session as a goodbye. This change keeps the gesture meaningful and doesn't draw too much attention rather than the game and interrupting the conversations and responses by slowing them down with gestures.

2.5 Conclusion (10%; Alfie)

GAZE implements multi-signal emotional inference across two independent modalities (*facial expression via WS-10 CNN and vocal emotion via WS-08 MLP*) alongside speech volume/RMS, response time, answer correctness, answer text, and derived temporal signals, hence yielding a more stable user-state estimate than any single channel. The hybrid architecture (*gpt-5.4* dialogue paired with symbolic rule-based inference and deterministic *gpt-4.1* answer verification) and adaptive game-switching directly target the novelty-decay assumption Smedegaard (2019, p. 414, Experiential novelty) reviews.

Every turn, `decide()` resolves a tone label (encouraging, celebratory, calm, energetic, neutral) from the inferred state; the LLM’s dialogue register thereby modulates live instead of holding a fixed persona, extending Tapus, Tapus and Mataric’s (2008, Abstract) personality-matching paradigm and Kahn et al.’s (2008, pp. 97-104) design patterns for sociality from static trait-fit to dynamic state-driven adaptation.

The multi-signal approach transfers to stroke rehabilitation re-engagement (Mataric et al., 2007, .pdf-p. 1, Abstract “*monitoring, encouragement, and reminders*”), educational tutoring, or neurodivergent support. GAZE sits within Section 1’s cognitive trajectory rather than the reactive social layer alone.

GAZE’s most important limitation is epistemic (*claim vs. proof*): Smedegaard’s (2019, p. 414, Experiential novelty) novelty-decay claim, central to the architectural rationale, is not herein empirically tested across multiple sessions; the adaptive engine *aims to* sustain engagement beyond novelty instead of demonstrating that it does. Furthermore, thresholds were derived from a single pilot, hence cross-user generalisation is unproven. The voice MLP, for its part, is the weakest channel by design, and may be quietly carrying inference noise that the face-primary rules cannot fully reject.

2.6. Task-4 References (5%)

2.6.1 Academic Books

- Laird, J. E. (2012) *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press. Available at: <https://www.scribd.com/document/264059670/The-Soar-Cognitive-Architecture-The-MIT-Press-2012-John-E-Laird> *FREE-TRIAL ACCESSED* (Accessed: 20 March 2026).

2.6.2 Academic Papers

- Ahn, M., Brohan, A., Brown, N., et al. (2022) ‘Do As I Can, Not As I Say: Grounding Language in Robotic Affordances’, in *Proceedings of the 6th Conference on Robot Learning (CoRL 2022), Proceedings of Machine Learning Research*, vol. 205. Available at: <https://arxiv.org/abs/2204.01691> (Accessed: 24 March 2026).
- Busso, C., Deng, Z., Yildirim, S., Bulut, M., Lee, C.M., Kazemzadeh, A., Lee, S., Neumann, U. and Narayanan, S. (2004) ‘Analysis of emotion recognition using facial expressions, speech and multimodal information’, in *Proceedings of the 6th International Conference on Multimodal Interfaces (ICMI ’04)*. New York: ACM, pp. 205–211. Available at: <https://dl.acm.org/doi/10.1145/1027933.1027968> (Accessed: 4 May 2026).
- Calvo, R.A. and D’Mello, S. (2010) ‘Affect detection: An interdisciplinary review of models, methods, and their applications’, *IEEE Transactions on Affective Computing*, 1(1), pp. 18–37. Available at: <https://doi.org/10.1109/t-affc.2010.1> (Accessed: 14 April 2026).
- Desai, M., Kaniarasu, P., Medvedev, M., Steinfeld, A. and Yanco, H. (2013) ‘Impact of robot failures and feedback on real-time trust’, in *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI ’13)*, pp. 251–258. Available at: <https://ieeexplore.ieee.org/document/6483596> (Accessed: 20 March 2026).
- Ekman, P. and Friesen, W.V. (1971) ‘Constants across cultures in the face and emotion’, *Journal of Personality and Social Psychology*, 17(2), pp. 124–129. Available at: <https://doi.org/10.1037/h0030377> (Accessed: 14 April 2026).
- El Ayadi, M., Kamel, M.S. and Karray, F. (2011) ‘Survey on speech emotion recognition: Features, classification schemes, and databases’, *Pattern Recognition*, 44(3), pp. 572–587. Available at: <https://doi.org/10.1016/j.patcog.2010.09.020> (Accessed: 14 April 2026).

- Garcez, A. d'A. and Lamb, L. C. (2023) 'Neurosymbolic AI: The 3rd Wave', *Artificial Intelligence Review*, 56, pp. 12387–12406. Available at: <https://link.springer.com/article/10.1007/s10462-023-10448-w> (Accessed: 20 March 2026).
- Ji, Z., Lee, N., Frieske, R., et al. (2023) 'Survey of Hallucination in Natural Language Generation', *ACM Computing Surveys*, 55(12), pp. 1–38. Available at: <https://dl.acm.org/doi/10.1145/3571730> (Accessed: 22 March 2026).
- Kahn, P.H., Jr., Freier, N.G., Kanda, T., Ishiguro, H., Ruckert, J.H., Severson, R.L. and Gill, B.T. (2008) 'Design Patterns for Sociality in Human-Robot Interaction', in *Proceedings of the 3rd ACM/IEEE International Conference on Human-Robot Interaction (HRI '08)*, pp. 97–104. Available at: <https://doi.org/10.1145/1349822.1349836> (Accessed: 14 April 2026).
- Li, S. and Deng, W. (2020) 'Deep Facial Expression Recognition: A Survey', *IEEE Transactions on Affective Computing*, 13(3), pp. 1195–1215. Available at: <https://doi.org/10.1109/TAFFC.2020.2981446> (Accessed: 14 April 2026).
- Matarić, M. J., Eriksson, J., Feil-Seifer, D. J. and Winstein, C. J. (2007) 'Socially assistive robotics for post-stroke rehabilitation', *Journal of NeuroEngineering and Rehabilitation*, 4(5), pp. 1–9. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC1821334/> (Accessed: 25 March 2026).
- Nikolaidis, S., Hsu, D. and Srinivasa, S. (2017) 'Human-robot mutual adaptation in collaborative tasks: Models and experiments', *The International Journal of Robotics Research*, 36(5–7), pp. 618–634. Available at: <https://journals.sagepub.com/doi/10.1177/0278364917690593> (Accessed: 20 March 2026).
- Picard, R.W. (1997) 'Affective Computing', *MIT Media Laboratory Perceptual Computing Section Technical Report No. 321*. Available at: <https://affect.media.mit.edu/pdfs/95.picard.pdf> (Accessed: 14 April 2026).
- Poria, S., Cambria, E., Bajpai, R. and Hussain, A. (2017) 'A review of affective computing: From unimodal analysis to multimodal fusion', *Information Fusion*, 37, pp. 98–125. Available at: <https://doi.org/10.1016/j.inffus.2017.02.003> (Accessed: 14 April 2026).
- Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2023) 'Robust Speech Recognition via Large-Scale Weak Supervision', in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, *Proceedings of Machine Learning Research*, 202, pp. 28492–28518. Available at: <https://proceedings.mlr.press/v202/radford23a/radford23a.pdf> (Accessed: 14 April 2026).
- Smedegaard, C. V. (2019) 'Reframing the Role of Novelty within Social HRI: From Noise to Information', in *Proceedings of the 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI '19)*, pp. 411–420. Available at: <https://dl.acm.org/doi/10.1109/HRI.2019.8673219> (Accessed: 22 March 2026).
- Spezialetti, M., Placidi, G. and Rossi, S. (2020) 'Emotion Recognition for Human-Robot Interaction: Recent Advances and Future Perspectives', *Frontiers in Robotics and AI*, 7, Article 532279. Available at: https://www.researchgate.net/publication/347520928_Emotion_Recognition_for_Human-Robot_Interaction_Recent_Advances_and_Future_Perspectives (Accessed: 14 April 2026).
- Tapus, A., Matarić, M. J. and Scassellati, B. (2007) 'Socially assistive robotics [Grand Challenges of Robotics]', *IEEE Robotics & Automation Magazine*, 14(1), pp. 35–42. Available at: <https://sczlab.yale.edu/sites/default/files/files/Tapus-RAM2007.pdf> (Accessed: 25 March 2026).
- Tapus, A., Țăpuș, C. and Matarić, M. J. (2008) 'User-robot personality matching and assistive robot behavior adaptation for post-stroke rehabilitation therapy', *Intelligent Service Robotics*, 1(2), pp. 169–183. Available at: <https://hal.science/hal-00770108/document> (Accessed: 26 March 2026).

3. Appendix

3.1 Video Demo

- YouTube link: <https://youtu.be/G9-pWalwjpa>

3.2 AI Declaration

Solo Work	S1 - Generative AI tools have not been used for this assessment.
Assisted Work	A1 – Idea Generation and Problem Exploration Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions). The interpretation and integration of research into the assignment remain the student's responsibility.
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.
	A11 - Other uses not listed above Please specify:
Partnered Work	P1 - Generative AI tool usage has been used integrally for this assessment Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:

Figure 5: Student Declaration of AI Tool use in this Assessment Table

I declare that I've used the AI tools listed below whilst preparing this assessment. I've read and understood the University of Plymouth's policy on the use of AI tools in assessment and confirm that my use falls within

the coursework's allowed categories, i.e. **A2 (Planning and Structuring Projects)** and **A4 (Research Assistance)**.

AI Tool Used	Purpose of Use	Extent of Use
ChatGPT	Outlining the project structure and drafting the report hierarchy (A2)	Initial structuring and final outline stages for GAZE
ChatGPT	Reviewing structural alignment against grading criteria and mapping word-count budgets (A2)	After and midway through section-drafting
ChatGPT	Finding relevant pages to read in the paper (A4)	Few times if the paper is too long
ChatGPT	General conversations via web-search AI about prevalent papers to read about how the topics relates to others' studies (A4)	Few times at the end
OpenAI API ('gpt-5.4' & 'gpt-4.1')	Core system functionality allowed via disclosure (proposal): generate dialogue, dynamic prompts, and verifying user answers within the GAZE Task-4 Novel Contribution	API used throughout local and lab testing sessions as the system's dialogue and reasoning engine

- ☒ I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.
- ☒ I confirm that all details provided above are an accurate description of how AI was used for this assessment.

3.3. Novel Python Project: .env and gaze22.1.py

3.3.1 .env:

```

1 OPENAI_API_KEY={get ur own key with: https://platform.openai.com/home}
2 NAO_IP=ROBOT_IP
3 GAZE_LOCAL_MODE=true # make true if no NAO connection; false is connected to Pepper (NAO)

```

3.3.2 gaze22.1.py:

```

1 """
2 GAZE: Game-Adaptive Zone of Engagement.
3
4 A Pepper (NAO) countdown-game host. The robot adapts difficulty,
5 pacing and feedback per turn from six signals (each paired with the function wherein it is
6   ↳ produced):
7   1- facial expression: (CNN, WS-10)    primary    --->    capture_and_classify() /
8       ↳ FacialExpressionModel.predict()
9   2- answer correctness: (performance) primary    --->    check_answer() (GPT-4.1 verifier)
10  3- response time:    (behavioural)    primary    --->    time.time() - question_start (main
11       ↳ loop)
12  4- verbal answer:    (Whisper text)    primary    --->    transcribe() (Whisper-1 verbose_json)
13  5- volume RMS:        (acoustic arousal) secondary --->    measure_volume() (RMS over the
14       ↳ canonical 16 kHz WAV)
15  6- vocal emotion:    (MLP, WS-08)      tie-breaker; MLP collapses to "fearful" on quiet/noisy
16       ↳ audio ---> classify_speech_emotion() --> SpeechEmotionModel.predict()
17  All six fan in to: AdaptiveEngine.infer_state() -> AdaptiveEngine.decide()
18
19 NOVELTY:
20 Multi-signal integration: single signals are not trusted alone. Voice is only consulted when the
21 face is Neutral, vocal-emotion confidence is >= 0.9 and the label is not "fearful"; the MLP might
22 collapse to it on quiet/noisy audio.

```

```

18
19 ARCHITECTURE: gpt-5.4 converse with access to four function-calling tools:
20   - generate_game_question (gpt-5.4 generates; validate_numbers_round stubbed return True)
21   - check_game_answer (gpt-4.1 yes/no verifier)
22   - evaluate_last_adaptation (self-evaluates previous round's strategy)
23   - request_more_time (acknowledges user's request for more think-time)
24
25 Initially used a wake-word defence but became unnecessary
26
27 `AdaptiveEngine`: six signals and adapt (change based on inference) difficulty,
    ↳ pacing/think-budget, and game-switching:
28   - gpt-4.1 for yes/no checks (check_answer, is_goodbye, resume-intent)
29   - gpt-5.4 for the main converse loop and question generation.
30
31 VARIOUS MODES:
32   1- LOCAL_MODE=true:
33     - webcam; works with Mac and Windows (no NAO connectionc)
34     - Mac/Windows mic
35     - local TTS; just run entirely without robot-connection because all input/output is local
        ↳ to tester's computer
36   2- LOCAL_MODE=false:
37     - Pepper over SSH;
38     - output (TTS, LEDs, gestures) on robot.
39
40   NAO-ran code must be non-indented to avoid processing issues with Pepper (Pepper's Python 2.7
        ↳ interpreter chokes on leading whitespace)
41
42   Now, everything is ran hybridly: dashboard stats, and the camera feed is streamed over TCP to
        ↳ the computer for
43   display and facial-expression inference. Pepper's camera is too slow for a responsive
        ↳ experience, and local webcam is much smoother. The microphone path is governed by
        ↳ LOCAL_MODE, but can be forced to the local machine with HYBRID_LOCAL_INPUT so the
        ↳ operator can speak from the computer whilst Pepper still does the talking.
44
45   3- HYBRID_LOCAL_INPUT (default = True):
46     - mic input is forced to the Mac despite LOCAL_MODE's preference
47     - essentially the brain works locally to offload inference and display whilst Pepper
        ↳ handles output (TTS, LEDs, gestures)
48
49 TESTING OBSERVATIONS:
50   - Pepper's onboard audio is *noisy*; Whisper (trained on podcasts, YouTube) therefore
        ↳ hallucinates on it and is gated by defences as follows:
51     - Silero VAD
52     - no_speech_prob, and a hallucination blacklist.
53
54   - Vosk wake-word ensure (has_wake_word()) was indeed wired early-in to
55   combat Whisper hallucinations on noisy Pepper audio. The
56   hybrid-required switch fixed this; local Mac audio sometimes
57   produces these, whereas Pepper's stream more-often does, hence
58   wake-word is now not necessary (transcribe(bypass_wake_word=True)).
59
60   - Pepper camera streaming (~10 fps) < local webcam
61   (~30 fps); the code therefore runs hybridly: length-prefixed RGB
62   over TCP port 9558 via pepper_video_loop/pepper_camera_receive_loop;
63   detect_thread_loop runs ~6-7 Hz; display repaints ~30 fps from
64   _preview_frame+_last_bbox.
65   - The emotion classifiers are lightweight, not clinical instruments and thus
66   robustness felt unfeasible
67

```

```

68
69 PER-PROPOSAL AUTHORSHIP:
70     Alfie: architecture, OpenAI integration, AdaptiveEngine. facial-expression pipeline,
71     Salman - game flow, gestures, LEDs, TTS pacing, save/load, testing.
72
73
74 LIVE PRE-DEMO CHECKLIST:
75 - [X] fix dashboard as it is just black when NAO
76 - [X] [X] eye colours should change
77 - [X] disregard all hallucinations until sufficient listened sentence
78 - [X] fix year (escape character hallucinations)
79 - [X] offload simpler tasks? to either computation or mini model
80 - [X] ensure facial expression inference 'logics commented sufficiently
81 - [X] ensure it notices and mitigates when user's disengaged
82 - [X] fix years being hallucinated
83 - [X] fix continuation from previous game 5 games to 3 make it says results every 3 rounds for
    ↪ example 2 out of 3 is correct and then adapts based in that
84
85 - [X] pivotise face over voice in multi-signal inference
86 - [X] ensure it saves all the time not just at least 5
87 times
88
89 - [X] refine the arm movement its too jittery
90 - [X] after escalate directly address user's disengaged
91 - [X] make it more random for more games as currently it keeps giving mainly the same answers even
    ↪ despite how hard it is
92 - [X] make voice very not important
93 - [X] make it ask for name straight away
94
95 - [X] configure audio frequency to work better maybe for the nao (this was later resolved by
    ↪ hybrid-local-input switch to use local mic
96
97 - [X] make dashboard camera FPS much more fluid because its too slow now; fix was to make it
    ↪ stream via TCP to computer
98
99 ""
100
101 import os, re, sys, json, time, types, wave, struct, socket, textwrap, tempfile, threading,
    ↪ subprocess, unicodedata
102 import tkinter as tk
103 from dataclasses import dataclass, field
104 from enum import Enum
105 from typing import Optional
106 print("[booting...] stdlib loaded", flush=True)
107
108 import numpy as np
109 import cv2
110 from PIL import Image, ImageTk
111 print("[booting...] numpy + opencv loaded", flush=True)
112
113 import paramiko
114 print("[booting...] paramiko loaded", flush=True)
115
116 import sounddevice as sd
117 import librosa
118 import soundfile as sf
119 # Vosk wake-word gate
120 try:
121     from vosk import Model as VoskModel, KaldiRecognizer

```

```

122     _vosk_import_ok = True
123 except ImportError:
124     VoskModel = None
125     KaldiRecognizer = None
126     _vosk_import_ok = False
127 # Silero VAD; pre-Whisper speech gate
128 try:
129     from silero_vad import load_silero_vad, read_audio, get_speech_timestamps
130     _silero_import_ok = True
131 except ImportError:
132     load_silero_vad = None
133     read_audio = None
134     get_speech_timestamps = None
135     _silero_import_ok = False
136 print("[booting...] audio stack loaded", flush=True)
137
138 from dotenv import load_dotenv
139 from openai import OpenAI
140 print("[booting...] openai loaded", flush=True)
141
142 import joblib
143 print("[booting...] loading tensorflow (this is slow on first run)...", flush=True)
144 import tensorflow as tf
145 from tensorflow.keras.models import model_from_json
146 print("[booting...] tensorflow loaded", flush=True)
147
148 # -----
149     ↪ -----
150 # Silero VAD loader; optional dep
151 _silero_model = None
152 if _silero_import_ok:
153     try:
154         _silero_model = load_silero_vad()
155         print("[booting] silero-vad loaded", flush=True)
156     except Exception as _vad_err:
157         print(f"[booting] silero-vad failed to load ({_vad_err}); VAD gate disabled", flush=True)
158
159 load_dotenv()
160
161
162 # NAO CONFIG
163 NAO_IP = os.getenv("NAO_IP", "ROBOT_IP")
164 NAO_USER = "nao"
165 NAO_PASS = "nao"
166 RECORD_MAX_SECS = 8 # ceiling (don't record longer than this)
167 RECORD_MIN_SECS = 1 # minimum recording before silence-detection
168 SILENCE_POLL_SECS = 0.25 # polling interval for silence detection on Pepper
169 SILENCE_DURATION = 1.2 # seconds of silence after speech to trigger stop
170 CALIBRATION_SECS = 3 # duration of start-up ambient noise calibration
171 ENERGY_BUFFER = 80 # margin above ambient baseline to set speech threshold
172 DEFAULT_ENERGY_THRESHOLD = 700 # fallback for if calibration fails
173 REMOTE_WAV = "/var/persistent/home/nao/input.wav"
174 REMOTE_IMG = "/var/persistent/home/nao/capture.jpg"
175 LOCAL_WAV = os.path.join(tempfile.gettempdir(), "gaze_input.wav")
176 LOCAL_IMG = os.path.join(tempfile.gettempdir(), "gaze_capture.jpg")
177 VOLUME_THRESHOLD = 700 # RMS; dashboard label only ("quiet" vs "normal"). Not a transcription
178     ↪ gate; see NAO_MIN_RMS_TO_TRANSCRIBE.
179 NAO_MIN_RMS_TO_TRANSCRIBE = 500 # near-empty WAV floor for the NAO-mode pre-transcribe gate;

```



```

    ↪ Silero + Whisper + blacklist do the real filtering downstream.
179 FACE_CONFIDENCE_THRESHOLD = 0.5 # below: face uncertain, treat neutral
180 VOICE_CONFIDENCE_THRESHOLD = 0.5 # threshold for vocal emotion is too uncertain; treat as neutral
181 SSH_TIMEOUT = 10
182 CMD_TIMEOUT = 60
183
184 # false when connected to pepper; true for testing when no Pepper's camera
185 USE_LOCAL_CAMERA = os.getenv("GAZE_LOCAL_CAMERA", "false").lower() == "true"
186
187 # uses local webcam; Mac microphone, and macOS TTS instead of Pepper-hardware
188 LOCAL_MODE = os.getenv("GAZE_LOCAL_MODE", "false").lower() == "true"
189 if LOCAL_MODE:
190     USE_LOCAL_CAMERA = True
191
192 # Hybrid: TTS, LEDs and gestures stay on Pepper whereas input (camera + mic) is local
193 HYBRID_LOCAL_INPUT = True
194 INPUT_IS_LOCAL = LOCAL_MODE or HYBRID_LOCAL_INPUT
195
196 DEBUG_PREVIEW = LOCAL_MODE or HYBRID_LOCAL_INPUT
197 _last_rms = 0.0 # shared with recording thread for overlay
198 _last_emotion = "" # updated by capture_and_classify
199
200 _preview_lock = threading.Lock()
201 _preview_state = {"emotion": "Neutral", "confidence": 0.0}
202 _preview_frame = None # latest RAW BGR frame (annotation now composited in camera_refresh)
203 _last_bbox = None # latest detected face bbox (x, y, w, h) in raw-frame coords; None if no face
204 DETECT_INTERVAL_SECS = 0.15
205
206 # pre-trained model paths; checks models/ first (portable), then workshop dir (dev fallback)
207 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
208 MODELS_DIR = os.path.join(SCRIPT_DIR, "models")
209 WORKSHOP_DIR = os.path.join(SCRIPT_DIR, "..", "..", "learning", "workshops") # go backwards
    ↪ to root of repo then go to learning/workshops
210
211 def find_model(local_name, workshop_subpath):
212     "Resolve a model local models/ dir first, then workshop fallback."
213     local = os.path.join(MODELS_DIR, local_name)
214     if os.path.exists(local):
215         return local
216     return os.path.join(WORKSHOP_DIR, workshop_subpath)
217
218 MODEL_JSON = find_model("model.json", os.path.join("[X]-facial-expression-detection",
    ↪ "model.json"))
219 MODEL_WEIGHTS = find_model("model_weights.weights.h5",
    ↪ os.path.join("[X]-facial-expression-detection", "model_weights.weights.h5"))
220 HAAR_CASCADE = find_model("haarcascade_frontalface_default.xml", os.path.join("[X]-ws-10",
    ↪ "haarcascade_frontalface_default.xml"))
221 SPEECH_MODEL = os.path.join(SCRIPT_DIR, "speech_emotion_model.pkl")
222 VOSK_MODEL_DIR = os.path.join(MODELS_DIR, "vosk-model-small-en-us-0.15")
223
224 # Vosk wake-word; "Pepper" or "Gaze"
225 _vosk_model = None
226 if _vosk_import_ok:
227     try:
228         _vosk_model = VoskModel(VOSK_MODEL_DIR)
229         print("[booting] vosk wake-word model loaded", flush=True)
230     except Exception as _vosk_err:
231         print(f"[booting] vosk failed to load ({_vosk_err}); wake-word gate disabled", flush=True)
232

```

```

233 RESPONSE_TIME_BASELINE = 15.0 # seconds; sits below 20s recording cap
234 CORRECTNESS_WINDOW = 5 # rolling window size
235 CORRECTNESS_FLOOR = 0.4 # below: ease off
236 CORRECTNESS_CEILING = 0.8 # above: ramp up
237 SILENCE_THRESHOLD = 2 # consecutive non-responses before intervention
238
239 SAVE_FILE = os.path.join(SCRIPT_DIR, "gaze_save.json")
240
241 # ensure AI key is set
242 if not os.getenv("OPENAI_API_KEY", "").strip():
243     raise SystemExit("ERROR: OPENAI_API_KEY not set. Add it to .env")
244 client = OpenAI()
245
246
247 class FacialExpressionModel:
248     "Pre-trained CNN: 7-classed emotion classifier (48x48 greyscale input)."
249
250     EMOTIONS = ["Angry", "Disgust", "Fear", "Happy", "Neutral", "Sad", "Surprise"]
251
252     def __init__(self, model_json_path, model_weights_path):
253         with open(model_json_path, "r") as f:
254             self.model = model_from_json(f.read())
255             self.model.load_weights(model_weights_path)
256             self.model.make_predict_function()
257
258     def predict(self, img):
259         "Return (emotion_label, confidence) from the (1, 48, 48, 1) array."
260         preds = self.model.predict(img, verbose=0)
261         idx = np.argmax(preds)
262         return self.EMOTIONS[idx], float(preds[0][idx])
263
264
265 class SpeechEmotionModel:
266     "Pre-trained MLP for vocal emotion classification (WS-08)."
267
268     EMOTIONS = ["calm", "happy", "fearful", "disgust"]
269
270     def __init__(self, model_path):
271         self.model = joblib.load(model_path)
272
273     @staticmethod # static because it's also used independently in classify_speech_emotion()
274     def extract_features(wav_path: str):
275         "Extract the same MFCC/chroma/mel feature vector used in WS-08 training."
276         with sf.SoundFile(wav_path) as sound_file:
277             audio = sound_file.read(dtype="float32")
278             sample_rate = sound_file.samplerate
279
280             if audio.ndim > 1:
281                 audio = audio.mean(axis=1)
282
283             # peak-normalise; helps quieter speakers
284             audio = librosa.util.normalize(audio)
285
286             n_fft = 2048
287             if len(audio) < n_fft:
288                 return None
289
290             stft = np.abs(librosa.stft(audio, n_fft=n_fft))
291

```

```

292     mfccs = np.mean(librosa.feature.mfcc(
293         y=audio, sr=sample_rate, n_mfcc=40).T, axis=0).flatten()
294
295     chroma = np.mean(librosa.feature.chroma_stft(
296         S=stft, sr=sample_rate).T, axis=0).flatten()
297
298     mel = np.mean(librosa.feature.melspectrogram(
299         y=audio, sr=sample_rate).T, axis=0).flatten()
300
301     return np.concatenate([mfccs, chroma, mel])
302
303     def predict(self, wav_path: str) -> tuple[str, float]:
304         "Return (emotion_label, confidence) from a WAV file."
305         features = self.extract_features(wav_path)
306         if features is None:
307             return "neutral", 0.0
308
309         features = features.reshape(1, -1)
310         label = self.model.predict(features)[0]
311         proba = self.model.predict_proba(features)[0]
312         confidence = float(np.max(proba))
313         return label, confidence
314
315     def classify_speech_emotion(speech_model, wav_path: str) -> tuple[str, float]:
316         "Classify the vocal emotion from a WAV file."
317         if speech_model is None:
318             return "neutral", 0.0
319         if INPUT_IS_LOCAL and not has_real_speech(wav_path):
320             return "neutral", 0.0
321         try:
322             return speech_model.predict(wav_path)
323         except Exception as e:
324             print(f"Speech emo classify failed: {e}; defaulting to neutral")
325             return "neutral", 0.0
326
327
328     class Difficulty(Enum):
329         EASY = 1
330         MEDIUM = 2
331         HARD = 3
332
333     class InferredState(Enum):
334         THRIVING = "thriving"
335         COMFORTABLE = "comfortable"
336         STRUGGLING = "struggling"
337         DISENGAGED = "disengaged" # thus re-engage
338         FRUSTRATED = "frustrated"
339
340     class GameType(Enum):
341         NUMBERS = "numbers"
342         LETTERS = "letters"
343
344     BASE_SYSTEM_PROMPT = """
345     You are GAZE: an stroke-assitive *social-companionistic* robot running on a NAO humanoid robot.
346     You are a therapeutic companion first, and an engaging game host second.
347
348     CONVERSATION GUIDELINES:
349     - Refer to yourself ONLY in the first person ('I', 'me', 'your companion').
350     - Have natural, flowing conversations with the user.

```

```

351 - You can play countdown-style games (numbers rounds and letters rounds) when the moment feels
    ↪ right or the user asks, but do NOT force a game every single turn.
352 - Each user message includes real-time emotional signals (facial expression, vocal emotion,
    ↪ volume, response time). Use these signals to adapt your tone and approach naturally; doN'T
    ↪ mention the signals explicitly.
353 - Keep responses concise: 2-3 sentences maximum. Your words are spoken aloud via text-to-speech,
    ↪ so brevity is essential.
354 - If a game is active, acknowledge the user's answer before moving on.
355 - If the user seems disengaged, try a different topic or suggest a game. (re-engage them)
356 - If the user asks for more time to think, call request_more_time and respond understandably.
357 - If the user says the game is too hard or too easy, adjust the difficulty naturally in your next
    ↪ generate_game_question call.
358 - Be genuinely present; you are the user's social companion.
359 """
360
361 # @dataclass: typed fields auto-become __init__ params
362
363 @dataclass
364 class GameState:
365     "Essentially tracks whether a countdown game is currently active."
366     active: bool = False
367     current_question: str = ""
368     current_answer: str = ""
369     category: str = ""
370     turn_count: int = 0
371     waiting: bool = False # user asked for more time to think
372     last_answer_checked: bool = False # was a game answer checked this turn?
373     last_answer_correct: bool = False # result of the last answer check
374     # snapshot of the answered round; survives a same-chain generate_game_question overwrite
375     answered_question: str = ""
376     answered_answer: str = ""
377     answered_game_type: Optional[GameType] = None
378     answered_difficulty: Optional[Difficulty] = None
379
380 @dataclass
381 class RoundResult:
382     "Record of a single game round."
383     round_number: int
384     game_type: GameType
385     difficulty: Difficulty
386     question: str
387     user_answer: str
388     correct: bool
389     response_time: float
390     facial_expression: str
391     expression_confidence: float
392     vocal_emotion: str
393     vocal_emotion_confidence: float
394     volume_rms: float # speech loudness (arousal signal)
395     inferred_state: InferredState
396     timestamp: float = field(default_factory=time.time)
397
398 @dataclass
399 class AdaptiveDecision:
400     "Output of the adaptive engine (what to do next)"
401     difficulty: Difficulty
402     game_type: GameType
403     inferred_state: InferredState
404     switch_game: bool

```

```

405     give_hint: bool
406     give_encouragement: bool
407     tone: str # "encouraging" / "celebratory" / "calm" / "energetic" / "neutral"
408
409
410 class AdaptiveEngine:
411     "Takes all six input signals and *infers* the user's real state. The adaptive engine also
412     ↪ evaluates if its previous adaptation worked, feeding that evaluation into the next
413     ↪ round's prompt."
414
415     def __init__(self):
416         self.history: list[RoundResult] = []
417         self.current_difficulty = Difficulty.MEDIUM
418         self.current_game = GameType.NUMBERS
419         self.consecutive_silences = 0
420         self.consecutive_correct = 0
421         self.consecutive_wrong = 0
422         self.games_played: dict[GameType, int] = {g: 0 for g in GameType}
423         self.game_switch_count = 0
424         self.adaptation_log: list[dict] = []
425         self.total_correct = 0
426         self.total_rounds_played = 0 # cumulative across resumed sessions
427         self.best_streak = 0
428         # recent-questions blocklist; cap 10
429         self.recent_questions: list[str] = []
430         self.recent_answers: list[str] = [] # answer-level dedup; catches mode-collapsed targets
431         ↪ even when gpt rephrases the question text
432         self.recent_game_types: list[str] = [] # game-type rotation hint; avoids 5-in-a-row Numbers
433         ↪ games
434         # adaptive think-budget; per-round baselines
435         self.think_budget_secs = float(RECORD_MAX_SECS) # hard ceiling
436         self.silence_tolerance_secs = float(SILENCE_DURATION) # post-speech silence
437         self.no_speech_max_secs = 5.0 # give up if no speech at all
438
439
440     @property
441     def round_number(self) -> int:
442         return len(self.history) + 1
443
444     def rolling_correctness(self) -> float:
445         recent = self.history[-CORRECTNESS_WINDOW:] # slice of 5 most recent rounds
446         if not recent:
447             return 0.5 # no data -> assume middle (neutral)
448         return sum(1 for r in recent if r.correct) / len(recent)
449
450
451     VOLUME_QUIET = 200 # below this -> low arousal (quiet/disengaged)
452     VOLUME_LOUD = 2000 # above this -> high arousal (excited/frustrated)
453
454     def infer_state(self, expression: str, response_time: float,
455                     correct: bool, answer_text: str,
456                     vocal_emotion: str = "neutral",
457                     vocal_conf: float = 0.0,
458                     volume_rms: float = 0.0) -> InferredState:
459         """
460         Infer the user's state from all signals.
461         Face is primary; voice is only derived when face is Neutral and
462         the voice signal is high-confidence and not "fearful" (the MLP
463         collapses to "fearful" in silence).

```

```

460     """
461     correctness = self.rolling_correctness()
462     clean = answer_text.strip().lower()
463     is_silent = (not clean or clean in {"i don't know", "skip", "pass", "next"}) # if no
        ↪ meaningful input || input matches a skip-command phrase in the set (set over list
        ↪ because it's faster) then indeed silent (True)

464
465     # Arousal bounds calibrated against ambient noise
466     high_arousal = volume_rms > self.VOLUME_LOUD
467     low_arousal = 0 < volume_rms < self.VOLUME_QUIET
468
469     # voice trusted only when not-fearful
470     trust_voice = (vocal_conf >= 0.9 and vocal_emotion != "fearful")
471
472     if is_silent:
473         self.consecutive_silences += 1
474     else:
475         self.consecutive_silences = 0
476     if correct:
477         self.consecutive_correct += 1
478         self.consecutive_wrong = 0
479     else:
480         self.consecutive_wrong += 1
481         self.consecutive_correct = 0
482
483     # 1- FACE-PRIMARY RULES
484
485     # thriving: good performance + fast responses
486     if (correctness >= CORRECTNESS_CEILING and response_time < RESPONSE_TIME_BASELINE * 0.5):
487         return InferredState.THRIVING
488     if expression == "Angry" and correct and response_time < RESPONSE_TIME_BASELINE * 0.6:
489         return InferredState.COMFORTABLE
490
491     # disengaged: silence + slow + poor performance
492     if self.consecutive_silences >= SILENCE_THRESHOLD:
493         return InferredState.DISENGAGED
494     if (expression == "Neutral"
495         and response_time > RESPONSE_TIME_BASELINE
496         and correctness < 0.5):
497         return InferredState.DISENGAGED
498     if (low_arousal and expression == "Neutral"
499         and response_time > RESPONSE_TIME_BASELINE * 0.8):
500         return InferredState.DISENGAGED
501
502     # frustrated: negative face + poor performance
503     if expression in ("Angry", "Disgust") and correctness < CORRECTNESS_FLOOR:
504         return InferredState.FRUSTRATED
505     if self.consecutive_wrong >= 3 and expression in ("Angry", "Sad", "Fear"):
506         return InferredState.FRUSTRATED
507     if (high_arousal and expression in ("Angry", "Disgust", "Fear")
508         and correctness < CORRECTNESS_FLOOR):
509         return InferredState.FRUSTRATED
510
511     # struggling: sadness + slow; poor correctness; fear + wrong
512     if expression == "Sad" and response_time > RESPONSE_TIME_BASELINE * 0.7:
513         return InferredState.STRUGGLING
514     if correctness < CORRECTNESS_FLOOR:
515         return InferredState.STRUGGLING
516     if expression == "Fear" and not correct:

```

```

517         return InferredState.STRUGGLING
518
519     # 2- voice tie-breakers; only when face neutral
520     if expression == "Neutral" and trust_voice:
521         if vocal_emotion == "happy" and correct and correctness >= CORRECTNESS_CEILING:
522             return InferredState.THRIVING
523         if vocal_emotion == "calm" and correctness >= 0.5:
524             return InferredState.COMFORTABLE
525
526     return InferredState.COMFORTABLE # default: face gave no negative signal, performance is
        ↪ holding
527
528     # core-decision function
529     def decide(self, expression: str, expression_conf: float,
530               response_time: float, correct: bool,
531               answer_text: str,
532               vocal_emotion: str = "neutral",
533               vocal_conf: float = 0.0,
534               volume_rms: float = 0.0) -> AdaptiveDecision:
535         "Return what to do next based on the inferred state."
536         state = self.infer_state(expression, response_time, correct, answer_text,
537                                vocal_emotion, vocal_conf=vocal_conf,
538                                volume_rms=volume_rms)
539         correctness = self.rolling_correctness()
540
541         new_difficulty = self.current_difficulty
542         new_game = self.current_game
543         switch_game = False
544         give_hint = False
545         give_encouragement = False
546         tone = "neutral"
547
548         if state == InferredState.THRIVING:
549             if self.current_difficulty != Difficulty.HARD:
550                 new_difficulty = Difficulty(self.current_difficulty.value + 1)
551                 tone = "energetic"
552             if self.consecutive_correct >= 3:
553                 give_encouragement = True # acknowledge streak
554
555         elif state == InferredState.COMFORTABLE:
556             if correctness > 0.7 and self.current_difficulty != Difficulty.HARD:
557                 new_difficulty = Difficulty(self.current_difficulty.value + 1)
558                 tone = "neutral"
559
560         elif state == InferredState.STRUGGLING:
561             if self.current_difficulty != Difficulty.EASY:
562                 new_difficulty = Difficulty(self.current_difficulty.value - 1)
563                 give_hint = True
564                 give_encouragement = True
565                 tone = "encouraging"
566
567         elif state == InferredState.FRUSTRATED:
568             new_difficulty = Difficulty.EASY
569             give_encouragement = True
570             tone = "calm"
571             if self.consecutive_wrong >= 3:
572                 switch_game = True
573                 new_game = self.pick_different_game()
574

```

```

575     elif state == InferredState.DISENGAGED:
576         tone = "energetic" # robot be engaging
577         give_encouragement = True
578         if self.consecutive_silences >= 3:
579             switch_game = True
580             new_game = self.pick_different_game()
581
582     self.current_difficulty = new_difficulty
583     if switch_game:
584         self.current_game = new_game
585         self.game_switch_count += 1
586
587     self.adaptation_log.append({
588         "round": self.round_number,
589         "state": state.value,
590         "action": {"difficulty": new_difficulty.name,
591                   "switch": switch_game,
592                   "hint": give_hint,
593                   "encouragement": give_encouragement},
594     })
595
596     return AdaptiveDecision(
597         difficulty=new_difficulty, game_type=self.current_game,
598         inferred_state=state, switch_game=switch_game,
599         give_hint=give_hint, give_encouragement=give_encouragement,
600         tone=tone,
601     )
602
603 def recommend_think_budget(self, state: InferredState, expression: str,
604                            prev_response_time: float,
605                            consecutive_silences: int, waiting: bool
606                            ) -> tuple[float, float, float]:
607     """
608     Choose how long to wait for the user this turn.
609     Slower or struggling users get more time, so a long pause does not
610     flip the system to "disengaged" too quickly.
611     Returns (no_speech_max, silence_secs, record_max_secs).
612     """
613     # baseline budget
614     no_speech_max = 5.0
615     silence_secs = float(SILENCE_DURATION)
616     record_max_secs = float(RECORD_MAX_SECS)
617
618     # round 1: stroke-recovery silence tolerance
619     if not self.history:
620         no_speech_max = max(no_speech_max, 7.0)
621         silence_secs = max(silence_secs, 2.5)
622         record_max_secs = max(record_max_secs, 15.0)
623
624     if state in (InferredState.STRUGGLING, InferredState.FRUSTRATED, InferredState.DISENGAGED):
625         no_speech_max = 8.0
626         silence_secs = 2.5
627         record_max_secs = 18.0
628
629     # graduated extension before disengaged flip
630     if consecutive_silences > 0:
631         no_speech_max = max(no_speech_max, 5.0 + consecutive_silences * 1.5)
632         silence_secs = max(silence_secs, 1.5 + consecutive_silences * 0.5)
633

```



```

634         if expression in ("Sad", "Fear"):
635             no_speech_max = max(no_speech_max, 7.0)
636             silence_secs = max(silence_secs, 2.0)
637
638         if prev_response_time > RESPONSE_TIME_BASELINE:
639             no_speech_max = max(no_speech_max, 7.0)
640             silence_secs = max(silence_secs, 2.0)
641
642         # LLM-flagged waiting: *honour* the signal but additively, so other cues stay weighted
643         if waiting:
644             no_speech_max += 3.0
645             silence_secs += 1.0
646             record_max_secs += 5.0
647
648         # defensive ceiling 20s; under CMD_TIMEOUT
649         record_max_secs = min(record_max_secs, 20.0)
650
651         self.no_speech_max_secs = no_speech_max
652         self.silence_tolerance_secs = silence_secs
653         self.think_budget_secs = record_max_secs
654
655         return no_speech_max, silence_secs, record_max_secs
656
657     def record_round(self, result: RoundResult):
658         self.history.append(result)
659         self.total_rounds_played += 1
660         self.games_played[result.game_type] = (
661             self.games_played.get(result.game_type, 0) + 1
662         )
663         if result.correct:
664             self.total_correct += 1
665         self.best_streak = max(self.best_streak, self.consecutive_correct)
666
667     def pick_different_game(self) -> GameType:
668         if self.current_game == GameType.NUMBERS:
669             return GameType.LETTERS
670         return GameType.NUMBERS
671
672     def get_session_summary(self) -> dict:
673         if not self.history:
674             return {"rounds": 0}
675         total = len(self.history)
676         correct = sum(1 for r in self.history if r.correct)
677         return {
678             "rounds": total,
679             "correct": correct,
680             "accuracy": round(correct / total, 2),
681             "avg_response_time": round(sum(r.response_time for r in self.history) / total, 1),
682             "games_played": {g.value: c for g, c in self.games_played.items() if c > 0},
683             "game_switches": self.game_switch_count,
684             "best_streak": self.best_streak,
685             "final_difficulty": self.current_difficulty.name,
686         }
687
688     # Self-evaluative adaptation
689     def evaluate_adaptation(self) -> Optional[str]:
690         "Evaluate whether the previous round's adaptation worked."
691         if len(self.adaptation_log) < 2 or len(self.history) < 2:
692             return None

```

```

693
694     prev_strategy = self.adaptation_log[-2]
695     curr_strategy = self.adaptation_log[-1]
696     prev_round = self.history[-2]
697     curr_round = self.history[-1]
698
699     prev_state = prev_strategy["state"]
700     curr_state = curr_strategy["state"]
701     prev_action = prev_strategy["action"]
702
703     evaluations = []
704
705     if prev_state in ("struggling", "frustrated"):
706         if curr_round.correct and not prev_round.correct:
707             evaluations.append("Previous adaptation WORKED: lowered difficulty and user answered
                                ↪ correctly this round (was incorrect before).")
708         elif not curr_round.correct:
709             evaluations.append("Previous adaptation DID NOT HELP YET: user still struggling
                                ↪ despite easier difficulty. Consider providing more support.")
710
711     # Did a difficulty increase overshoot for a thriving user?
712     if prev_state == "thriving" and prev_action["difficulty"] == "HARD":
713         if curr_round.correct:
714             evaluations.append("Previous adaptation WORKED: increased difficulty and user is
                                ↪ still performing well.")
715         elif not curr_round.correct:
716             evaluations.append("Previous adaptation OVERSHOT: increased difficulty but user got
                                ↪ it wrong. Might need to ease back.")
717
718     # Did a re-engagement attempt work for a disengaged user?
719     if prev_state == "disengaged":
720         if curr_state != "disengaged":
721             evaluations.append(f"Previous adaptation WORKED: user was disengaged but is now
                                ↪ {curr_state}. Re-engagement WAS effective.")
722         else:
723             evaluations.append("Previous adaptation DID NOT HELP: user remains disengaged. Try a
                                ↪ different approach or switch game type.")
724
725     if prev_action.get("switch"):
726         if curr_state in ("thriving", "comfortable"):
727             evaluations.append(
728                 "Game switch WORKED: user transitioned to a positive state."
729             )
730         elif curr_state in ("struggling", "frustrated", "disengaged"):
731             evaluations.append(
732                 "Game switch DID NOT HELP: user is still in a negative state."
733             )
734
735     if (prev_action.get("encouragement")
736         and prev_state in ("struggling", "frustrated")
737         and curr_round.response_time < prev_round.response_time):
738         evaluations.append(
739             "Encouragement appears effective: user responded faster this round."
740         )
741
742     if not evaluations:
743         return None
744
745     return ("Adaptation evaluation from previous round:\n"

```

```

746         + "\n".join(f"- {e}" for e in evaluations))
747
748
749 GAME_DESCRIPTIONS = {
750     GameType.NUMBERS: """
751     a Countdown-style numbers round. Pick SIX numbers by sampling from {1 to 100}; vary the mix
752     ↪ each round so no two rounds in a row feel identical.
753     Pick a TARGET in the range 50-999 (anywhere in that range, NOT always around 100), reachable
754     ↪ from the chosen six via +, -, *, /.
755     The user must combine the given numbers with those operators to reach the target. Each number
756     ↪ may be used at most once and it should be humanly solveable""",
757     GameType.LETTERS: """
758     a Countdown-style letters round: give the user a set of 9 random letters (a mix of vowels and
759     ↪ consonants; vary the letter set every round) and ask them to form the longest word
760     ↪ possible using only those letters.
761     Each letter can only be used once""",
762 }
763
764
765 DIFFICULTY_DESCRIPTIONS = {
766     Difficulty.EASY: "easy (straightforward, common knowledge, single-step)",
767     Difficulty.MEDIUM: "medium (requires some thought, moderately specific)",
768     Difficulty.HARD: "hard (obscure, multi-step, requires deep knowledge)",
769 }
770
771 # SSH AND PEPPER ROBOT HELPERS: NON-INDENTED PYTHON STRINGS
772 # (adapted from lab-robot-code-fin.py i.e. when
773 # we tested OpenAI on the NAO robot perhaps too early)
774
775 def ssh_connect():
776     "Open SSH connection to Pepper and return the client."
777     ssh = paramiko.SSHClient()
778     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
779     ssh.connect(NAO_IP, username=NAO_USER, password=NAO_PASS, timeout=SSH_TIMEOUT)
780     return ssh
781
782 def nao_run(ssh, code):
783     "Execute a Python 2 snippet on Pepper via SSH."
784     escaped = code.replace("'", "'\\'")
785     try:
786         _, stdout, _ = ssh.exec_command(f"python -c '{escaped}'", timeout=CMD_TIMEOUT)
787         return stdout.read().decode().strip()
788     except Exception as e:
789         print(f"Pepper SSH exec_command dropped: {e}")
790         return ""
791
792 #listens silent
793 #not triggered by noise
794 def nao_calibrate_ambient(ssh) -> int:
795     "Calibrate the mic energy threshold to the room."
796     try:
797         # col 0 only; Pepper rejects indented python -c
798         raw = nao_run(ssh, f"")
799     from naoqi import ALProxy
800     import time
801
802     audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
803     samples = []
804     start = time.time()
805     while (time.time() - start) < {CALIBRATION_SECS}:

```

```

800     # all four mics; side speakers else missed
801     try:
802         front = audio.getFrontMicEnergy()
803         left = audio.getLeftMicEnergy()
804         right = audio.getRightMicEnergy()
805         rear = audio.getRearMicEnergy()
806         samples.append(max(front, left, right, rear))
807     except Exception:
808         # older firmwares may expose only getFrontMicEnergy; fall back gracefully
809         samples.append(audio.getFrontMicEnergy())
810     time.sleep(0.2)
811
812 if samples:
813     avg = sum(samples) / len(samples)
814     print(int(avg))
815 else:
816     print(0)
817 """
818     ambient = int(raw) if raw.strip().isdigit() else 0
819     threshold = ambient + ENERGY_BUFFER
820     print(f" Ambient energy: {ambient}, speech threshold: {threshold}")
821     return threshold
822 except Exception as e:
823     print(f" Calibration failed ({e}), using default threshold: {DEFAULT_ENERGY_THRESHOLD}")
824     return DEFAULT_ENERGY_THRESHOLD
825
826 def nao_record(ssh, energy_threshold: int = DEFAULT_ENERGY_THRESHOLD,
827               record_max_secs: float = RECORD_MAX_SECS,
828               silence_secs: float = SILENCE_DURATION):
829     """Record audio on Pepper with dynamic silence detection.
830     - get robot's front microphone (getFrontMicEnergy) to stop recording early if silence
831       ↳ detected
832     - calibrated energy threshold to avoid false positives from ambient noise
833     - if getFrontMicEnergy is unsupported (e.g. older firmware), fall back to a safe
834       ↳ fixed-duration recording to ensure the demo still works, albeit without silence
835       ↳ detection
836     """
837     # fixed-duration fallback for old firmware
838     nao_run(ssh, f"""
839 from naoqi import ALProxy
840 import time
841
842 rec = ALProxy("ALAudioRecorder", "127.0.0.1", 9559)
843
844 rec.stopMicrophonesRecording()
845 rec.startMicrophonesRecording("{REMOTE_WAV}", "wav", 16000, [1, 1, 1, 1])
846
847 try:
848     audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
849
850     speech_detected = False
851     silence_start = None
852     start = time.time()
853     threshold = {energy_threshold}
854
855     while True:
856         elapsed = time.time() - start
857
858         # hard ceiling; never exceed max duration

```

```

856         if elapsed >= {record_max_secs}:
857             break
858
859         # all four mics; side speakers else missed
860         try:
861             energy = max(
862                 audio.getFrontMicEnergy(),
863                 audio.getLeftMicEnergy(),
864                 audio.getRightMicEnergy(),
865                 audio.getRearMicEnergy(),
866             )
867         except Exception:
868             energy = audio.getFrontMicEnergy() # firmware fallback
869
870         if elapsed < {RECORD_MIN_SECS}:
871             # minimum recording period
872             if energy > threshold:
873                 speech_detected = True
874                 time.sleep({SILENCE_POLL_SECS})
875                 continue
876
877             if energy > threshold:
878                 speech_detected = True
879                 silence_start = None
880             else:
881                 if speech_detected and silence_start is None:
882                     silence_start = time.time()
883                 if speech_detected and silence_start is not None:
884                     if (time.time() - silence_start) >= {silence_secs}:
885                         break
886
887             time.sleep({SILENCE_POLL_SECS})
888
889     except Exception as e:
890         # firmware fallback; if getFrontMicEnergy() unsupported, fixed-duration recording keeps the
891         ↪ demo alive
892         print(" [Silence detection failed: " + str(e) + "] Falling back to fixed-duration recording")
893         time.sleep({record_max_secs})
894     rec.stopMicrophonesRecording()
895     """
896     sftp = ssh.open_sftp()
897     sftp.get(REMOTE_WAV, LOCAL_WAV)
898     sftp.close()
899     # no gain stage; amplified room noise
900
901     # WAV layout diagnostic; four-mic recording may land multi-channel
902     try:
903         with wave.open(LOCAL_WAV, "rb") as wf:
904             secs = wf.getnframes() / float(wf.getframerate())
905             print(f" WAV debug: channels={wf.getnchannels()}, rate={wf.getframerate()},
906                 ↪ frames={wf.getnframes()}, secs={secs:.2f}")
907     except Exception as e:
908         print(f" WAV debug failed: {e}")
909
910     # collapse to mono 16 kHz so downstream gates see a canonical layout
911     force_mono_16k_wav(LOCAL_WAV)
912     def force_mono_16k_wav(path: str) -> None:

```

```

913     "Convert `path` to mono 16 kHz int16 PCM."
914     try:
915         audio, sr = sf.read(path, dtype="float32")
916         if audio.size == 0:
917             return
918         if audio.ndim > 1:
919             # loudest mic; mics not phase-aligned
920             rms_by_channel = np.sqrt(np.mean(audio.astype(np.float64) ** 2, axis=0))
921             audio = audio[:, int(np.argmax(rms_by_channel))]
922         if sr != LOCAL_SAMPLE_RATE:
923             audio = librosa.resample(audio, orig_sr=sr, target_sr=LOCAL_SAMPLE_RATE)
924         audio = np.clip(audio, -1.0, 1.0)
925         sf.write(path, audio, LOCAL_SAMPLE_RATE, subtype="PCM_16")
926     except Exception as e:
927         print(f" Mono conversion skipped ({e})")
928
929     #responses arrive in one block no splitting
930     #delivered with no pauses
931     def split_into_sentences(text: str) -> list[str]:
932         "Split dialogue into sentences for speech delivery."
933         raw_segments = re.split(r'(?<=[.!?])\s+', text.strip())
934         sentences = []
935         for seg in raw_segments:
936             for line in seg.split("\n"):
937                 cleaned = line.strip()
938                 if cleaned:
939                     sentences.append(cleaned)
940         return sentences if sentences else [text.strip()]
941
942     _TTS_REPLACEMENTS = { # prevent unicode characters from breaking Pepper Text-To-Speech
943         '"': "'", "'": '"',
944         '""': "'", "'": '""',
945         "-": "-", "-": "-", "-": "-",
946         "...": "...", " ": " ",
947     }
948
949     def clean_for_tts(text: str) -> str:
950         "Strip gesture tags and Unicode escapes for Pepper TTS."
951         text = text or ""
952         text = re.sub(r'\[gesture:\w+\]', '', text)
953         text = unicodedata.normalize("NFKC", text)
954         for bad, good in _TTS_REPLACEMENTS.items():
955             text = text.replace(bad, good)
956         # animated-speech tags
957         text = re.sub(r'\^[A-Za-z]+(?:\[([~])*\])?', '', text)
958         # ALTextToSpeech tags
959         text = re.sub(r'\\[A-Za-z]{2,8}=[^\\]*\\', '', text)
960         # literal \uXXXX leftovers
961         text = re.sub(r'\\u[0-9a-fA-F]{4}', '', text)
962         # control characters
963         text = re.sub(r'[\x00-\x1f\x7f-\x9f]', ' ', text)
964         return re.sub(r'\s+', ' ', text).strip()
965
966     #single ssh calls
967     def nao_say(ssh, text):
968         "Speak text on Pepper with sentence-level pausing."
969         text = clean_for_tts(text)
970         sentences = split_into_sentences(text)
971         safe_sentences = json.dumps(sentences)

```

```

972
973     nao_run(ssh, f"""
974 from naoqi import ALProxy
975 import time
976
977 try:
978     tts = ALProxy("ALTextToSpeech", "127.0.0.1", 9559)
979     sentences = {safe_sentences}
980
981     for i, sentence in enumerate(sentences):
982         tts.say(sentence)
983         if i < len(sentences) - 1:
984             time.sleep(0.4)
985 except Exception as e:
986     print(" [TTS failed] " + str(e))
987 """
988
989 #animations with speech
990 def nao_say_animated(ssh, text):
991     "Try animated speech; fall back to plain TTS."
992     safe = json.dumps(text)
993     try:
994         nao_run(ssh, f"""
995 from naoqi import ALProxy
996 ALProxy("ALAnimatedSpeech","127.0.0.1",9559).say({safe})
997 """
998     except Exception as e:
999         print(f" Animated speech failed mid-sentence: {e}")
1000         nao_say(ssh, text)
1001
1002 def nao_capture_image(ssh):
1003     "Capture a photo from Pepper's camera and download it. Kept as a fallback for one-off stills;
1004     ↪ the live dashboard now uses pepper_video_loop()."
1005     nao_run(ssh, f"""
1006 from naoqi import ALProxy
1007 pc = ALProxy("ALPhotoCapture","127.0.0.1",9559)
1008 pc.setResolution(2)
1009 pc.setPictureFormat("jpg")
1010 pc.takePicture("{os.path.dirname(REMOTE_IMG)}/",
1011     ↪ "{os.path.splitext(os.path.basename(REMOTE_IMG))[0]}")
1012 """
1013     sftp = ssh.open_sftp()
1014     sftp.get(REMOTE_IMG, LOCAL_IMG)
1015     sftp.close()
1016
1017 PEPPER_VIDEO_PORT = 9558
1018 _pepper_video_channel = None
1019
1020 def pepper_video_loop(ssh):
1021     """New persistent ALVideoDevice on Pepper streaming length-prefixed RGB frames over
1022     TCP (Transmission Control Protocol)"""
1023     global _pepper_video_channel
1024
1025     code = textwrap.dedent('''
1026         from naoqi import ALProxy
1027         import socket, struct, time
1028
1029         HOST = "0.0.0.0"
1030         PORT = {port}

```

```

1029
1030     cam = ALProxy("ALVideoDevice", "127.0.0.1", 9559)
1031     # camera_id=0 (top), resolution=1 (320x240), colour_space=11 (RGB), fps=10
1032     name = cam.subscribeCamera("gaze_video", 0, 1, 11, 10)
1033
1034     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
1035     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
1036     server.bind((HOST, PORT))
1037     server.listen(1)
1038
1039     conn, addr = server.accept()
1040
1041     try:
1042         while True:
1043             img = cam.getImageRemote(name)
1044             if img is None:
1045                 time.sleep(0.05)
1046                 continue
1047             width = img[0]
1048             height = img[1]
1049             data = img[6]
1050             payload = struct.pack("!II", width, height) + data
1051             conn.sendall(struct.pack("!I", len(payload)) + payload)
1052             time.sleep(0.1)
1053         finally:
1054             try:
1055                 cam.unsubscribe(name)
1056             except Exception:
1057                 pass
1058             try:
1059                 conn.close()
1060             except Exception:
1061                 pass
1062             server.close()
1063     '').format(port=PEPPER_VIDEO_PORT)
1064
1065     escaped = code.replace("'", "'\\'")
1066     # async; nao_run() reads stdout, would otherwise block
1067     _stdin, stdout, _stderr = ssh.exec_command(f"python -c '{escaped}'")
1068     _pepper_video_channel = stdout.channel # keep referenced so Paramiko doesn't GC the channel
1069     ↪ out from under the remote process
1070
1071 def _recv_exact(sock, n):
1072     "Receive exactly n bytes; raise if the socket closes mid-frame."
1073     data = b""
1074     while len(data) < n:
1075         packet = sock.recv(n - len(data))
1076         if not packet:
1077             raise ConnectionError("Socket closed whilst receiving Pepper camera frame")
1078         data += packet
1079     return data
1080
1081
1082 def pepper_camera_receive_loop(pepper_ip: str):
1083     "Pull length-prefixed RGB frames off Pepper:9558 into _preview_frame for detect_thread_loop to
1084     ↪ consume; ten 1-second connect-retries because Pepper-side bind() needs a beat to land
1085     ↪ after exec_command."
1086     global _preview_frame

```



```

1085
1086     sock = None
1087     for attempt in range(10):
1088         try:
1089             sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
1090             sock.connect((pepper_ip, PEPPER_VIDEO_PORT))
1091             print(f" Pepper video socket connected on attempt {attempt + 1}.")
1092             break
1093         except (ConnectionRefusedError, OSError) as e:
1094             print(f" Pepper video connect attempt {attempt + 1} failed ({e}); retrying...")
1095             try:
1096                 sock.close()
1097             except Exception:
1098                 pass
1099             sock = None
1100             time.sleep(1.0)
1101     if sock is None:
1102         print(" Pepper video stream unreachable; dashboard will stay black.")
1103         return
1104
1105     try:
1106         while True: # whilst running
1107             size_raw = _recv_exact(sock, 4)
1108             size = struct.unpack("I", size_raw)[0]
1109             payload = _recv_exact(sock, size)
1110             width, height = struct.unpack("II", payload[:8])
1111             rgb_bytes = payload[8:]
1112             rgb = np.frombuffer(rgb_bytes, dtype=np.uint8).reshape((height, width, 3))
1113             frame = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR) # rest of pipeline is BGR (OpenCV
                  ↪ convention)
1114             with _preview_lock:
1115                 _preview_frame = frame
1116         except Exception as e:
1117             print(f" Pepper video receiver loop exited: {e}")
1118     finally:
1119         try:
1120             sock.close()
1121         except Exception:
1122             pass
1123
1124 def nao_track_face(ssh, enable=True):
1125     "Toggle face tracking on Pepper."
1126     try:
1127         if enable:
1128             nao_run(ssh, """
1129 from naoqi import ALProxy
1130 ALProxy("ALFaceDetection","127.0.0.1",9559).subscribe("gaze_face")
1131 t = ALProxy("ALTracker","127.0.0.1",9559)
1132 t.registerTarget("Face", 0.1)
1133 t.track("Face")
1134 """)
1135         else:
1136             nao_run(ssh, """
1137 from naoqi import ALProxy
1138 t = ALProxy("ALTracker","127.0.0.1",9559)
1139 t.stopTracker()
1140 t.unregisterAllTargets()
1141 try:
1142     ALProxy("ALFaceDetection","127.0.0.1",9559).unsubscribe("gaze_face")

```

```

1143 except Exception as e:
1144     print(" [Face unsubscribe failed] " + str(e))
1145     """)
1146     except Exception as e:
1147         print(f" Face tracking unavailable: {e}")
1148
1149 #leds
1150 def nao_set_leds(ssh, group, colour, duration=1.0):
1151     try:
1152         nao_run(ssh, f"""
1153 from naoqi import ALProxy
1154 ALProxy("ALLeds", "127.0.0.1", 9559).fadeRGB("{group}", {colour}, {duration})
1155 """)
1156     except Exception as e:
1157         print(f" LED set ignored: {e}")
1158
1159 # LOCAL MODE HELPERS
1160
1161 LOCAL_SAMPLE_RATE = 16000 # Whisper expects 16 kHz; we resample from native rate
1162
1163 LOCAL_SILENCE_RMS = 40 # default RMS; overridden by local_calibrate_ambient()
1164 _local_speech_detected = False # set by local_record(); used as transcription gate
1165 LOCAL_SILENCE_SECS = 1.2 # seconds of post-speech silence to stop recording; 1.5 → 1.2
1166     ↪ (aphasia-safe)
1167 LOCAL_MIN_SECS = 1.0 # minimum recording before silence detection kicks in
1168 LOCAL_NO_SPEECH_MAX = 3.0 # stop if no speech detected at all after this many seconds; 5.0 → 3.0
1169     ↪ (dominant lag)
1170 LOCAL_ENERGY_BUFFER = 60 # margin above ambient baseline for speech detection; 50 → 25 to catch
1171     ↪ quiet speech
1172
1173 def local_calibrate_ambient() -> int:
1174     "Calibrate the local-testing (Mac) mic's ambient noise level; mirrors nao_calibrate_ambient()
1175     ↪ so LOCAL_MODE testing behaves like the robot."
1176
1177     global LOCAL_SILENCE_RMS
1178     dev_info = sd.query_devices(kind="input")
1179     dev_index = dev_info["index"]
1180     native_rate = int(dev_info["default_samplerate"])
1181     chunk_size = int(native_rate * 0.2)
1182     samples = []
1183
1184     print(f"Calibrating Mac mic (stay quiet for {CALIBRATION_SECS}s)...")
1185
1186     def cb(indata, frames, time_info, status): # get stable baseline from average RMS
1187         rms = (np.mean(indata.astype(np.float64) ** 2)) ** 0.5
1188         samples.append(rms)
1189
1190     with sd.InputStream(samplerate=native_rate, channels=1, dtype="int16",
1191                        blocksize=chunk_size, device=dev_index, callback=cb):
1192         time.sleep(CALIBRATION_SECS)
1193
1194     if samples:
1195         ambient = int(sum(samples) / len(samples))
1196     else:
1197         ambient = 0
1198
1199     threshold = ambient + LOCAL_ENERGY_BUFFER
1200     LOCAL_SILENCE_RMS = threshold
1201
1202     # 500 was too high for MacBook mic

```

```

1198     global VOLUME_THRESHOLD
1199     VOLUME_THRESHOLD = max(LOCAL_SILENCE_RMS * 4, 100)
1200
1201     # scale to room; static thresholds confuse loud/quiet rooms
1202     AdaptiveEngine.VOLUME_QUIET = max(ambient * 2, 200) # 200 = absolute quiet floor
1203     AdaptiveEngine.VOLUME_LOUD = max(ambient * 10, 2000)
1204
1205     print(f" Ambient RMS: {ambient}, silence threshold: {threshold}, transcription gate:
           ↪ {VOLUME_THRESHOLD}")
1206     print(f" Arousal thresholds: QUIET={AdaptiveEngine.VOLUME_QUIET},
           ↪ LOUD={AdaptiveEngine.VOLUME_LOUD}")
1207     return threshold
1208
1209 def local_record(max_secs: float = RECORD_MAX_SECS,
1210                 no_speech_max: float = LOCAL_NO_SPEECH_MAX,
1211                 silence_secs: float = LOCAL_SILENCE_SECS):
1212     "Record audio from the Mac's built-in microphone to LOCAL_WAV with silence detection mirroring
       ↪ Pepper's dynamic recording."
1213     # select the default input device
1214     dev_info = sd.query_devices(kind="input")
1215     dev_index = dev_info["index"]
1216     native_rate = int(dev_info["default_samplerate"])
1217     sd.default.device = (dev_index, None)
1218     chunk_size = int(native_rate * SILENCE_POLL_SECS)
1219
1220     buffer = []
1221     speech_detected = False
1222     silence_start = None
1223     elapsed = 0.0
1224
1225     print(f" Recording from Mac mic (up to {max_secs}s, stops after silence)...")
1226
1227     def callback(indata, frames, time_info, status):
1228         buffer.append(indata.copy())
1229
1230     for attempt in range(2): #one retry if PortAudio error (if the device briefly is unavailable
       ↪ after Pepper usage)
1231         try:
1232             with sd.InputStream(samplerate=native_rate, channels=1, dtype="int16",
1233                               blocksize=chunk_size, callback=callback):
1234                 while elapsed < max_secs:
1235                     time.sleep(SILENCE_POLL_SECS)
1236                     elapsed += SILENCE_POLL_SECS
1237
1238                     if not buffer:
1239                         continue
1240                     data = buffer[-1]
1241
1242                     rms = (np.mean(data.astype(np.float64) ** 2)) ** 0.5 # compute RMS of the latest
       ↪ chunk for real-time feedback
1243                     global _last_rms
1244                     _last_rms = rms # ' ' (U+2593 DARK SHADE) and ' ' (U+2591
       ↪ LIGHT SHADE) extracted from Unicode 1.1 Block Elements
1245                     print(f"\r [{elapsed:.1f}s] RMS: {rms:.0f} {' ' if rms > LOCAL_SILENCE_RMS else
       ↪ ' '}", end="", flush=True)
1246
1247                     if elapsed < LOCAL_MIN_SECS:
1248                         if rms > LOCAL_SILENCE_RMS:
1249                             speech_detected = True

```

```

1250             continue
1251
1252             if not speech_detected and elapsed >= no_speech_max:
1253                 break
1254
1255             if rms > LOCAL_SILENCE_RMS:
1256                 speech_detected = True
1257                 silence_start = None
1258             else:
1259                 if speech_detected and silence_start is None:
1260                     silence_start = elapsed
1261                 if speech_detected and silence_start is not None:
1262                     if (elapsed - silence_start) >= silence_secs:
1263                         break
1264             break # stream ran to completion
1265         except sd.PortAudioError as e:
1266             print(f"\n [PortAudio error, attempt {attempt + 1}] {e}")
1267             if attempt == 0:
1268                 time.sleep(0.5) # let CoreAudio release the device
1269                 continue
1270             print(" Mic unavailable; saving silent recording and continuing.")
1271
1272     print()
1273
1274     if buffer:
1275         audio_native = np.concatenate(buffer).flatten().astype(np.float32) / 32768.0
1276         # resample to 16 kHz for Whisper
1277         audio_16k = librosa.resample(audio_native, orig_sr=native_rate, target_sr=LOCAL_SAMPLE_RATE)
1278         audio_int16 = (audio_16k * 32768.0).astype(np.int16)
1279     else:
1280         audio_int16 = np.zeros((0,), dtype=np.int16)
1281
1282     # speech-detected flag so the transcription gate can use it
1283     global _local_speech_detected
1284     _local_speech_detected = speech_detected
1285
1286     with wave.open(LOCAL_WAV, "wb") as wf:
1287         wf.setnchannels(1)
1288         wf.setsampwidth(2)
1289         wf.setframerate(LOCAL_SAMPLE_RATE)
1290         wf.writeframes(audio_int16.tobytes())
1291     print(f" Recording saved ({elapsed:.1f}s, speech={'yes' if speech_detected else 'no'})")
1292
1293 def local_say(text: str):
1294     "Speak text using host OS built-in TTS (macOS `say` or Windows SAPI)."
1295     text = clean_for_tts(text)
1296     try:
1297         if sys.platform == "win32":
1298             # Windows: SAPI via PowerShell; env-var avoids escaping
1299             subprocess.run(
1300                 ["powershell", "-NoProfile", "-Command",
1301                  "Add-Type -AssemblyName System.Speech;(New-Object"
1302                  "    ↪ System.Speech.Synthesis.SpeechSynthesizer).Speak($env:GAZE_TTS_TEXT)"],
1303                 env={**os.environ, "GAZE_TTS_TEXT": text},
1304                 check=True, timeout=30,
1305             )
1306         else:
1307             subprocess.run(["say", text], check=True, timeout=30)
1308     except Exception as e:

```

```

1308         print(f" Local TTS broke: {e}")
1309
1310     def say(ssh_tts, text):
1311         "Dispatch TTS to local or Pepper depending on mode."
1312         text = clean_for_tts(text)
1313         if LOCAL_MODE:
1314             local_say(text)
1315         else:
1316             nao_say(ssh_tts, text)
1317         time.sleep(0.5) # ensure text-to-speech (TTS) drains so it does not hear itself
1318
1319     def record(ssh, energy_threshold,
1320               no_speech_max: float = LOCAL_NO_SPEECH_MAX,
1321               silence_secs: float = LOCAL_SILENCE_SECS,
1322               record_max_secs: float = RECORD_MAX_SECS):
1323         """Dispatch recording to local or Pepper depending on mode;
1324         per-turn think-budget set by: AdaptiveEngine.recommend_think_budget().
1325         INPUT_IS_LOCAL routes the mic to the Mac even when LOCAL_MODE is False
1326         (gaze21 hybrid: Pepper-out, Mac-in)."""
1327         if INPUT_IS_LOCAL:
1328             local_record(max_secs=record_max_secs,
1329                          no_speech_max=no_speech_max,
1330                          silence_secs=silence_secs)
1331         else:
1332             nao_record(ssh, energy_threshold,
1333                       record_max_secs=record_max_secs,
1334                       silence_secs=silence_secs)
1335
1336     # gesture mapping; motions per game/emotional context
1337     # tags: celebrate, encourage, think, wave, calm, energetic, neutral
1338
1339     GESTURE_CODE = {
1340         "wave": """
1341 from naoqi import ALProxy
1342 import time
1343 m = ALProxy("ALMotion","127.0.0.1",9559)
1344 m.setStiffnesses("RArm", 1.0)
1345 safe = ["RShoulderPitch","RShoulderRoll","RElbowYaw","RElbowRoll","RWristYaw","RHand"]
1346 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1347 time.sleep(0.2)
1348 m.angleInterpolation(["RShoulderRoll"], [-0.45], [1.5], True)
1349 time.sleep(0.2)
1350 m.angleInterpolation(safe, [-0.3, -0.3, 1.0, 1.0, 0.0, 1.0], [2.0]*6, True)
1351 for _ in range(3):
1352     m.angleInterpolation(["RWristYaw"], [0.5], [1.2], True)
1353     m.angleInterpolation(["RWristYaw"], [-0.5], [1.2], True)
1354 time.sleep(0.4)
1355 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1356 m.setStiffnesses("RArm", 0.0)
1357 """,
1358
1359         "correct_wave": """
1360 from naoqi import ALProxy
1361 import time
1362 m = ALProxy("ALMotion","127.0.0.1",9559)
1363 m.setStiffnesses("RArm", 1.0)
1364 safe = ["RShoulderPitch","RShoulderRoll","RElbowYaw","RElbowRoll","RWristYaw","RHand"]
1365 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1366 time.sleep(0.2)

```

```

1367 m.angleInterpolation(["RShoulderRoll"], [-0.45], [1.5], True)
1368 time.sleep(0.2)
1369 m.angleInterpolation(safe, [-0.3, -0.3, 1.0, 1.0, 0.0, 1.0], [2.0]*6, True)
1370 for _ in range(3):
1371     m.angleInterpolation(["RWristYaw"], [0.5], [1.2], True)
1372     m.angleInterpolation(["RWristYaw"], [-0.5], [1.2], True)
1373 time.sleep(0.4)
1374 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1375 m.setStiffnesses("RArm", 0.0)
1376 """
1377 }
1378
1379
1380 # LED COLOUR MAP
1381 LED_COLOURS = {
1382     InferredState.THRIVING: 0x0000FF00, # green
1383     InferredState.COMFORTABLE: 0x0000FFFF, # cyan
1384     InferredState.STRUGGLING: 0x00FFFF00, # yellow
1385     InferredState.FRUSTRATED: 0x00FF0000, # red
1386     InferredState.DISENGAGED: 0x00FF00FF, # magenta
1387 }
1388
1389
1390 _STATE_COLOURS = {
1391     InferredState.THRIVING: "green",
1392     InferredState.COMFORTABLE: "blue",
1393     InferredState.STRUGGLING: "orange",
1394     InferredState.FRUSTRATED: "red",
1395     InferredState.DISENGAGED: "grey",
1396 }
1397
1398 def nao_gesture(ssh, gesture_type: str):
1399     "Execute a gesture on Pepper aligned to the game context."
1400     code = GESTURE_CODE.get(gesture_type, GESTURE_CODE["wave"])
1401     try:
1402         nao_run(ssh, code)
1403     except Exception as e:
1404         print(f" Gesture {gesture_type!r} did not play: {e}")
1405
1406
1407 def measure_volume() -> float:
1408     "RMS amplitude of the WAV; soundfile-based so multi-channel files collapse to mono cleanly."
1409     try:
1410         audio, _sr = sf.read(LOCAL_WAV, dtype="float32")
1411         if audio.size == 0:
1412             return 0.0
1413         if audio.ndim > 1:
1414             audio = audio.mean(axis=1)
1415         return float(np.sqrt(np.mean((audio * 32768.0) ** 2)))
1416     except Exception as e:
1417         print(f" Volume RMS calc failed: {e}")
1418         return 0.0
1419
1420 # Whisper hallucination blacklist; pre-normalised
1421 # exact-match: short fillers that would false-positive if matched as substrings.
1422 WHISPER_HALLUCINATION_EXACT = {
1423     "you", "mm", "hmm", "uh", "um",
1424     "thank you", "thanks",
1425     "

```

```

1426     "    ",
1427     "    ",
1428     "    ",
1429     "    ",
1430 }
1431
1432 # fragments: if any appears anywhere in the normalised text, treat as hallucination.
1433 WHISPER_HALLUCINATION_FRAGMENTS = (
1434     "questions or comments",
1435     "questions or other problems",
1436     "post them in the comments",
1437     "in the comments section",
1438     "in the comment section",
1439     "if you like this video",
1440     "if you liked this video",
1441     "if you enjoyed the video",
1442     "if you enjoyed this video",
1443     "please subscribe",
1444     "subscribe and like",
1445     "like and subscribe",
1446     "like it and subscribe",
1447     "subscribe to our channel",
1448     "subscribe to my channel",
1449     "dont forget to subscribe",
1450     "do not forget to subscribe",
1451     "hit the like button",
1452     "smash that like button",
1453     "click the link",
1454     "link in the description",
1455     "link in the bio",
1456     "in the description below",
1457     "thanks for watching",
1458     "thank you for watching",
1459     "thank you so much for watching",
1460     "thanks so much for watching",
1461     "see you next time",
1462     "ill see you next time",
1463     "see you in the next video",
1464     "see you in the next one",
1465     # Whisper-prompt regurgitation; default prompt mentions "companion robot"
1466     "companion robot",
1467     "chats with a companion",
1468     "answers a quiz",
1469     "with a companion robot",
1470 )
1471
1472 # back-compat alias
1473 WHISPER_HALLUCINATIONS = WHISPER_HALLUCINATION_EXACT
1474
1475 def normalise_for_blacklist(text: str) -> str:
1476     "Normalise so blacklist matches independent of Whisper output."
1477     t = unicodedata.normalize("NFKC", text).lower()
1478     kept = [ch for ch in t if ch.isalnum() or ch.isspace()]
1479     return " ".join("".join(kept).split())
1480
1481 # regex blacklist; URL outros, promo boilerplate
1482 _URL_HALLUCINATION =
1483     ↪ re.compile(r"\bhttps?://|www\.\b|w+\.(?:com|org|net|au|co\.uk|google|sites)\b", re.I)
1483 _DISCLAIMER_HALLUCINATION = re.compile(r"\b(please see|visit|subscribe|like and

```

```

1484         ↪ subscribe|disclaimer|description)\b.{0,40}\b(complete|full|link|below|above)\b", re.I)
1485 def is_known_hallucination(text: str) -> bool:
1486     norm = normalise_for_blacklist(text)
1487     if norm == "" or norm in WHISPER_HALLUCINATION_EXACT:
1488         return True
1489     # any fragment present anywhere in the normalised text counts
1490     if any(frag in norm for frag in WHISPER_HALLUCINATION_FRAGMENTS):
1491         return True
1492     # structural catch: Whisper leaks YouTube-description URLs on silence-amplified input
1493     if _URL_HALLUCINATION.search(text):
1494         return True
1495     if _DISCLAIMER_HALLUCINATION.search(text):
1496         return True
1497     return False
1498
1499 def has_real_speech(wav_path: str, min_speech_ms: int = 250,
1500                    threshold: float = 0.5) -> bool:
1501     "Silero VAD pre-gate; relaxed defaults so short answers (yes/Tom/six) survive."
1502     if _silero_model is None or get_speech_timestamps is None:
1503         return True
1504     try:
1505         wav = read_audio(wav_path, sampling_rate=LOCAL_SAMPLE_RATE)
1506         segments = get_speech_timestamps( # pass in the audio and model to return a list of dicts
1507             ↪ containing the start and end frame indices of genuine speech
1508             wav, _silero_model,
1509             sampling_rate=LOCAL_SAMPLE_RATE,
1510             min_speech_duration_ms=min_speech_ms,
1511             threshold=threshold,
1512             return_seconds=False,
1513         )
1514         return bool(segments)
1515     except Exception as e:
1516         print(f" Silero VAD check failed ({e}); falling through to Whisper")
1517         return True
1518
1519 def has_wake_word(wav_path: str) -> bool:
1520     """
1521     Vosk wake-word gate. True if "Pepper" or "Gaze" is heard in the WAV.
1522     Returns True if the Vosk model didn't load (optional dep).
1523     """
1524     if _vosk_model is None or KaldiRecognizer is None:
1525         return True
1526     try:
1527         rec = KaldiRecognizer(_vosk_model, LOCAL_SAMPLE_RATE,
1528                               json.dumps(["pepper", "gaze", "[unk]"]))
1529         with wave.open(wav_path, "rb") as wf:
1530             while True:
1531                 data = wf.readframes(4000)
1532                 if len(data) == 0:
1533                     break
1534                 rec.AcceptWaveform(data)
1535             final = json.loads(rec.FinalResult())
1536             text = (final.get("text") or "").lower()
1537             return "pepper" in text or "gaze" in text
1538     except Exception as e:
1539         print(f" Vosk wake-word check failed ({e}); falling through to Whisper")
1540         return True
1541

```



```

1541 #open ai whisperings and incase fails and in silent
1542 def transcribe(bypass_wake_word: bool = False,
1543               whisper_prompt: str = "User answers a quiz or chats with a companion robot.",
1544               record_again=None,
1545               max_hallucination_retries=None) -> str:
1546     # Silero VAD -> wake-word -> Whisper -> hallucination blacklist
1547     attempts_remaining = max_hallucination_retries
1548     while True:
1549         if INPUT_IS_LOCAL and not _local_speech_detected:
1550             return ""
1551
1552         # Silero VAD hard gate; NAO + LOCAL
1553         if not has_real_speech(LOCAL_WAV):
1554             print(" Silero VAD found no speech; skipping Whisper.")
1555             return ""
1556
1557         # Vosk wake-word gate; bypass for name prompt
1558         if not bypass_wake_word and not has_wake_word(LOCAL_WAV):
1559             print(" No wake-word detected; skipping Whisper.")
1560             return ""
1561
1562     try:
1563         with open(LOCAL_WAV, "rb") as fh:
1564             resp = client.audio.transcriptions.create(
1565                 model="whisper-1", #
1566                 file=fh, #
1567                 response_format="verbose_json", # get self-signals for hallucination detection
1568                 temperature=0.0, # zero randomness
1569                 prompt=whisper_prompt,
1570                 timeout=API_TIMEOUT,
1571             )
1572             text = (getattr(resp, "text", "") or "").strip()
1573             print(f" Whisper raw text: {text!r}")
1574
1575             # Whisper self-signals from verbose_json
1576             segments = getattr(resp, "segments", None) or []
1577             suspected_hallucination = False
1578             if segments:
1579                 no_speech_vals = [s.no_speech_prob for s in segments
1580                                 if getattr(s, "no_speech_prob", None) is not None]
1581                 logprob_vals = [s.avg_logprob for s in segments
1582                                if getattr(s, "avg_logprob", None) is not None]
1583                 compression_vals = [s.compression_ratio for s in segments
1584                                    if getattr(s, "compression_ratio", None) is not None]
1585                 print(f" Whisper diagnostics: no_speech={no_speech_vals},
1586                       ↪ avg_logprob={logprob_vals}, compression={compression_vals}")
1587                 if no_speech_vals and max(no_speech_vals) > 0.6:
1588                     print(f" Whisper flagged silence (max no_speech_prob={max(no_speech_vals):.2f});
1589                           ↪ dropping {text!r}")
1590                     return ""
1591                 if logprob_vals and min(logprob_vals) < -1.3:
1592                     print(f" Whisper low-confidence (min avg_logprob={min(logprob_vals):.2f});
1593                           ↪ dropping {text!r}")
1594                     return ""
1595                 # repetition-loop hallucination; flag for retry path
1596                 if compression_vals and max(compression_vals) > 2.4:
1597                     print(f" Whisper repetition loop (max
1598                           ↪ compression_ratio={max(compression_vals):.2f}); flagging {text!r} as
1599                           ↪ hallucination")

```

```

1595         suspected_hallucination = True
1596
1597     # normalised hallucination blacklist + Whisper-self-signal hallucinations
1598     if suspected_hallucination or is_known_hallucination(text):
1599         print(f" Filtered Whisper hallucination: {text!r}")
1600         if record_again is not None and (attempts_remaining is None or attempts_remaining >
1601             ↪ 0):
1602             if attempts_remaining is not None: # whilst more retries remain decrement each
1603                 ↪ time-taken and re-record
1604                 attempts_remaining -= 1
1605                 print(f" Disregarding hallucination; listening again (attempts left:
1606                     ↪ {attempts_remaining}).")
1607             else:
1608                 print(f" Disregarding hallucination; listening again.")
1609                 record_again()
1610                 continue
1611         return ""
1612
1613     # Strip leading "Pepper"/"Gaze" so handlers receive just the answer; \b blocks
1614     ↪ "Pepperoni"/"Gazebo" false positives..
1615     stripped = re.sub(r'(?i)^\s*(pepper|gaze)\b[.\s]*', '', text).strip()
1616     return stripped
1617 except Exception as e:
1618     print(f" Whisper transcribe failed ({e}); returning empty")
1619     return ""
1620
1621 # facial-expression pipeline
1622
1623 def capture_thread_loop(camera):
1624     """Tight capture loop; just grab the latest frame.
1625     Decoupled from detection so the dashboard runs at native fps."""
1626     global _preview_frame
1627     while True:
1628         ret, frame = camera.read()
1629         if not ret: # if camera read fails keep old frame as preview
1630             time.sleep(0.03)
1631             continue
1632         with _preview_lock:
1633             _preview_frame = frame
1634
1635 def detect_thread_loop(face_model, face_cascade):
1636     """Run cascade + CNN on a downscaled copy of the latest frame; update cached bbox + emotion.
1637     6-7 Hz is enough for a turn-based consumer."""
1638     global _last_emotion, _last_bbox
1639     while True:
1640         time.sleep(DETECT_INTERVAL_SECS)
1641         with _preview_lock:
1642             frame = _preview_frame
1643         if frame is None:
1644             continue
1645
1646         # 2x downscale; 4x faster cascade
1647         small = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)
1648         gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
1649         faces = face_cascade.detectMultiScale(gray, 1.3, 5)
1650
1651         if len(faces) == 0:
1652             bbox = None
1653             emotion, conf = "Neutral", 0.0

```

```

1650     else:
1651         x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
1652         roi = gray[y:y+h, x:x+w]
1653         resized = cv2.resize(roi, (48, 48))
1654         inp = resized[np.newaxis, :, :, np.newaxis]
1655         emotion, conf = face_model.predict(inp)
1656         # scale 'bbox' back to raw-frame coordinates (was shrunk 2x)
1657         bbox = (x*2, y*2, w*2, h*2)
1658
1659     with _preview_lock:
1660         _preview_state["emotion"] = emotion
1661         _preview_state["confidence"] = conf
1662         _last_bbox = bbox
1663         _last_emotion = emotion
1664
1665 def start_preview_thread(camera, face_model, face_cascade):
1666     "Start both the capture and the detection daemon threads."
1667     cap_t = threading.Thread(target=capture_thread_loop,
1668                             args=(camera,),
1669                             daemon=True)
1670     det_t = threading.Thread(target=detect_thread_loop,
1671                             args=(face_model, face_cascade),
1672                             daemon=True)
1673     cap_t.start()
1674     det_t.start()
1675     return cap_t, det_t
1676
1677 def capture_and_classify(ssh, face_model, face_cascade,
1678                         local_camera=None) -> tuple[str, float]:
1679     "Return (emotion, confidence). Local-camera branch reads + classifies inline; NAO branch reads
1680     ↪ cached state set by detect_thread_loop off the live Pepper video stream."
1681     # local-camera + preview thread already running: just read the cache
1682     if DEBUG_PREVIEW and local_camera is not None:
1683         with _preview_lock:
1684             return _preview_state["emotion"], _preview_state["confidence"]
1685
1686     # NAO: read cached state from detect thread
1687     if local_camera is None:
1688         with _preview_lock:
1689             return _preview_state["emotion"], _preview_state["confidence"]
1690
1691     # local-camera per-turn classification (DEBUG_PREVIEW disabled)
1692     ret, frame = local_camera.read()
1693     if not ret:
1694         print(" [Local camera failed] cv2.VideoCapture.read() returned False")
1695         return "Neutral", 0.0
1696
1697     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
1698     faces = face_cascade.detectMultiScale(gray, 1.3, 5)
1699
1700     if len(faces) == 0:
1701         bbox = None
1702         emotion, conf = "Neutral", 0.0
1703     else:
1704         x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
1705         roi = gray[y:y+h, x:x+w]
1706         resized = cv2.resize(roi, (48, 48))
1707         inp = resized[np.newaxis, :, :, np.newaxis]
1708         emotion, conf = face_model.predict(inp)

```

```

1708         bbox = (x, y, w, h)
1709
1710     global _preview_frame, _last_bbox
1711     with _preview_lock:
1712         _preview_state["emotion"] = emotion
1713         _preview_state["confidence"] = conf
1714         _last_bbox = bbox
1715         _preview_frame = frame
1716
1717     return emotion, conf
1718
1719
1720 API_TIMEOUT = 10 # 10-second timeout; prevents Pepper-freeze if OpenAI/network stalls
1721
1722 def check_answer(user_answer: str, correct_answer: str,
1723                 question_context: str) -> bool:
1724     "Verify the user's answer via gpt."
1725     if not user_answer.strip():
1726         return False
1727
1728     try:
1729         resp = client.chat.completions.create(
1730             model="gpt-4.1",
1731             messages=[{
1732                 "role": "system",
1733                 "content": "You are an answer checker. Given a question, the correct answer, and the
1734                     ↳ user's spoken answer, determine if the user is correct. Be lenient with
1735                     ↳ pronunciation, phrasing, and partial answers that demonstrate knowledge.
1736                     ↳ Respond with ONLY 'correct' or 'incorrect'.",
1737             }, {
1738                 "role": "user",
1739                 "content": f"""Question: {question_context}
1740                     Correct answer: {correct_answer}
1741                     User's answer: {user_answer}""",
1742             }],
1743             temperature=0.0,
1744             timeout=API_TIMEOUT,
1745         )
1746         verdict = resp.choices[0].message.content.strip().lower()
1747         return verdict == "correct" # if AI says its 'correct' return verdict=correct
1748     except Exception as e:
1749         print(f" Answer verifier API failed: {e}")
1750         fallback_answer = correct_answer.lower().strip()
1751         fallback_user = user_answer.lower().strip()
1752
1753         return fallback_user == fallback_answer
1754
1755 # save/load sessions
1756
1757 def save_session(engine: AdaptiveEngine, preferred_game: Optional[GameType] = None,
1758                 quiet: bool = False):
1759     """Save session progress so user can continue later.
1760     quiet=True suppresses the "Session saved to ..." print, used when saving
1761     after every round so the log doesn't fill up with save confirmations.
1762     """
1763     data = {
1764         "total_correct": engine.total_correct,
1765         "total_rounds_played": engine.total_rounds_played,
1766         "best_streak": engine.best_streak,

```

```

1764     "games_played": {g.value: c for g, c in engine.games_played.items()},
1765     "game_switches": engine.game_switch_count,
1766     "last_difficulty": engine.current_difficulty.value,
1767     "last_game": engine.current_game.value,
1768     "preferred_game": preferred_game.value if preferred_game else None,
1769     "rounds_played": len(engine.history),
1770     "recent_questions": engine.recent_questions[-30:],
1771     "recent_answers": engine.recent_answers[-30:],
1772     "recent_game_types": engine.recent_game_types[-30:],
1773     "round_log": [
1774         {
1775             "round": r.round_number,
1776             "game": r.game_type.value,
1777             "difficulty": r.difficulty.value,
1778             "correct": r.correct,
1779             "time": round(r.response_time, 1),
1780             "expression": r.facial_expression,
1781             "vocal": r.vocal_emotion,
1782             "state": r.inferred_state.value,
1783         }
1784         for r in engine.history
1785     ],
1786 }
1787 with open(SAVE_FILE, "w") as f:
1788     json.dump(data, f, indent=2)
1789 if not quiet:
1790     print(f" Session saved to {SAVE_FILE}")
1791
1792 def load_session() -> Optional[dict]:
1793     "Load a previously saved session, if one exists."
1794     if not os.path.exists(SAVE_FILE):
1795         return None
1796     try:
1797         with open(SAVE_FILE, "r") as f:
1798             return json.load(f)
1799     except (json.JSONDecodeError, IOError) as e:
1800         print(f" Save file corrupt, ignoring: {e}")
1801         return None
1802
1803 def restore_engine(save_data: dict) -> AdaptiveEngine:
1804     "Restore engine state from saved data."
1805     engine = AdaptiveEngine()
1806     engine.total_correct = save_data.get("total_correct", 0)
1807     # legacy fallback; older saves used rounds_played instead
1808     engine.total_rounds_played = save_data.get("total_rounds_played",
1809                                                save_data.get("rounds_played", 0))
1810     engine.best_streak = save_data.get("best_streak", 0)
1811     engine.game_switch_count = save_data.get("game_switches", 0)
1812     engine.current_difficulty = Difficulty(save_data.get("last_difficulty", 2))
1813     engine.current_game = GameType(save_data.get("last_game", "numbers"))
1814     engine.recent_questions = list(save_data.get("recent_questions", []))[-30:]
1815     engine.recent_answers = list(save_data.get("recent_answers", []))[-30:]
1816     engine.recent_game_types = list(save_data.get("recent_game_types", []))[-30:]
1817     for g_val, count in save_data.get("games_played", {}).items():
1818         try:
1819             engine.games_played[GameType(g_val)] = count
1820         except ValueError as e:
1821             print(f" Could not restore game type {g_val!r}: {e}")
1822     return engine

```

```

1823
1824 def delete_save():
1825     "Remove the save file after a completed session or on user request."
1826     if os.path.exists(SAVE_FILE):
1827         os.remove(SAVE_FILE)
1828
1829 # OpenAI function-calling + conversation helpers
1830 '''
1831     - generate_game_question: called by the LLM when it wants to ask a new question; returns a
1832       ↪ question + correct answer for the specified game type and difficulty
1833     - check_game_answer: called by the LLM after the user answers a question; returns answer
1834       ↪ correctness if so
1835     - evaluate_last_adaptation: called by the LLM to self-evaluate if the previous round's
1836       ↪ adaptive strategy did indeed help the user
1837     - request_more_time: called by the LLM when the user asks for more time
1838 '''
1839 TOOLS = [
1840     {
1841         "type": "function",
1842         "function": {
1843             "name": "generate_game_question",
1844             "description": "Generate a new countdown-style game question. Call this when the
1845               ↪ conversation naturally leads to playing a game.",
1846             "parameters": {
1847                 "type": "object",
1848                 "properties": {
1849                     "game_type": {
1850                         "type": "string",
1851                         "enum": ["numbers", "letters"],
1852                         "description": "Type of countdown game round.",
1853                     },
1854                     "difficulty": {
1855                         "type": "string",
1856                         "enum": ["EASY", "MEDIUM", "HARD"],
1857                         "description": "Difficulty level for the question.",
1858                     },
1859                 },
1860                 "required": ["game_type", "difficulty"],
1861             },
1862         },
1863     },
1864     {
1865         "type": "function",
1866         "function": {
1867             "name": "check_game_answer",
1868             "description": "Verify whether the user's spoken answer to the current game question is
1869               ↪ correct. Call this after the user gives an answer to an active game question.",
1870             "parameters": {
1871                 "type": "object",
1872                 "properties": {
1873                     "user_answer": {
1874                         "type": "string",
1875                         "description": "What the user said.",
1876                     },
1877                     "correct_answer": {
1878                         "type": "string",
1879                         "description": "The known correct answer.",
1880                     },
1881                 },
1882                 "required": ["user_answer", "correct_answer"],
1883             },
1884         },
1885     },
1886 ]

```

```

1877         },
1878         "question_context": {
1879             "type": "string",
1880             "description": "The original question text for context.",
1881         },
1882     },
1883     "required": ["user_answer", "correct_answer", "question_context"],
1884 },
1885 },
1886 },
1887 {
1888     "type": "function",
1889     "function": {
1890         "name": "evaluate_last_adaptation",
1891         "description": "Self-evaluate whether the previous round's adaptive strategy actually
1892             ↪ helped the user. Returns a natural-language evaluation.",
1893         "parameters": {"type": "object", "properties": {}},
1894     },
1895 },
1896 {
1897     "type": "function",
1898     "function": {
1899         "name": "request_more_time",
1900         "description": "The user has asked for more time to think about the current game
1901             ↪ question. Acknowledge their request warmly.",
1902         "parameters": {"type": "object", "properties": {}},
1903     },
1904 },
1905 ]
1906
1907 def build_signal_context(engine: AdaptiveEngine,
1908     expression: str, expr_conf: float,
1909     vocal_emo: str, vocal_conf: float,
1910     vol_rms: float, response_time: float) -> str:
1911     "Build a signal summary string injected alongside every user message so the LLM can adapt its
1912         ↪ behaviour to the user's real-time emotional state without explicit adaptive-engine
1913         ↪ instructions."
1914     correctness = engine.rolling_correctness()
1915     recent_faces = [r.facial_expression for r in engine.history[-3:]]
1916     recent_vocal = [r.vocal_emotion for r in engine.history[-3:]]
1917
1918     # think-budget (float) -> (str) word label for LLM
1919     if engine.think_budget_secs >= 17.0:
1920         pacing = "relaxed and patient"
1921     elif engine.think_budget_secs <= 13.0:
1922         pacing = "brisk and energetic"
1923     else:
1924         pacing = "standard"
1925
1926     lines = [
1927         "--- LIVE SIGNALS ---",
1928         f"Turn: {engine.round_number}",
1929         f"Face: {expression} ({expr_conf:.0%}) [PRIMARY signal: trust this first]",
1930         f"Voice: {vocal_emo} ({vocal_conf:.0%}) [advisory only; the vocal model is noisy and often
1931             ↪ stuck on 'fearful']",
1932         f"Volume: {vol_rms:.0f} RMS",
1933         f"Response time: {response_time:.1f}s",
1934         f"Rolling accuracy (last {CORRECTNESS_WINDOW}): {correctness:.0%}", # overall correctness
1935         f"System pacing: {pacing}",

```

```

1931     ]
1932     if recent_faces:
1933         lines.append(f"Recent faces: {'', '.join(recent_faces)}")
1934     if recent_vocal:
1935         lines.append(f"Recent vocal: {'', '.join(recent_vocal)}")
1936
1937     return "\n".join(lines)
1938
1939 def converse(conversation: list, tools: list) -> object:
1940     "General-conversation OpenAI `gpt-5.4` chat-completion call with function calling."
1941     try:
1942         resp = client.chat.completions.create(
1943             model="gpt-5.4",
1944             messages=conversation,
1945             tools=tools,
1946             temperature=0.8, # bit of randomness/creativity for more interesting companion
1947             timeout=API_TIMEOUT,
1948         )
1949         return resp.choices[0].message
1950     except Exception as e:
1951         print(f"converse() API failed... : {e}")
1952         return types.SimpleNamespace(
1953             content="I had a brief network hiccup. Let's keep going! [gesture:think]",
1954             tool_calls=None, role="assistant")
1955
1956 def execute_tool_call(tool_name: str, tool_args: dict,
1957                       engine: AdaptiveEngine, game_state: GameState,
1958                       conversation: list,
1959                       preferred_game: Optional[GameType],
1960                       dashboard=None) -> str:
1961     "Dispatch a function-calling tool invocation and return a JSON string result."
1962     if tool_name == "generate_game_question":
1963         gt = tool_args.get("game_type", "numbers")
1964         diff = tool_args.get("difficulty", "MEDIUM")
1965         result = generate_game_question_internal(
1966             gt, diff,
1967             recent=engine.recent_questions,
1968             recent_answers=engine.recent_answers,
1969             recent_game_types=engine.recent_game_types,
1970         )
1971         # dedup: question + answer + game_type
1972         new_q = result.get("question", "").strip()
1973         new_a = str(result.get("answer", "")).strip()
1974         if new_q:
1975             engine.recent_questions.append(new_q)
1976             engine.recent_questions = engine.recent_questions[-30:]
1977         if new_a:
1978             engine.recent_answers.append(new_a)
1979             engine.recent_answers = engine.recent_answers[-30:]
1980         engine.recent_game_types.append(gt)
1981         engine.recent_game_types = engine.recent_game_types[-30:]
1982         # sync adaptive engine with LLM's chosen difficulty so the engine's next decide() starts
1983             ↪ from correct baseline
1984     try:
1985         engine.current_difficulty = Difficulty[diff]
1986     except (KeyError, ValueError):
1987         pass
1988     # also sync current_game; record_round logs game_type from this
1989     try:

```



```

1989         engine.current_game = GameType(gt)
1990     except ValueError:
1991         pass
1992     game_state.active = True
1993     game_state.current_question = result.get("question", "")
1994     game_state.current_answer = result.get("answer", "")
1995     game_state.category = result.get("category", "")
1996     return json.dumps(result)
1997
1998 elif tool_name == "check_game_answer":
1999     ua = tool_args.get("user_answer", "")
2000     if not game_state.active:
2001         game_state.last_answer_checked = False
2002         return json.dumps({"correct": False, "error": "no active game question"})
2003     # snapshot before same-chain regen overwrites
2004     game_state.answered_question = game_state.current_question
2005     game_state.answered_answer = game_state.current_answer
2006     game_state.answered_game_type = engine.current_game
2007     game_state.answered_difficulty = engine.current_difficulty
2008     # Python state is the source of truth, not LLM-supplied args
2009     is_correct = check_answer(
2010         ua,
2011         game_state.current_answer,
2012         game_state.current_question,
2013     )
2014     # last_answer_checked: ensures record_round() runs
2015     game_state.last_answer_checked = True
2016     game_state.last_answer_correct = is_correct
2017     if is_correct:
2018         game_state.active = False
2019     return json.dumps({"correct": is_correct})
2020
2021 elif tool_name == "evaluate_last_adaptation":
2022     evaluation = engine.evaluate_adaptation()
2023     return json.dumps({"evaluation": evaluation})
2024
2025 elif tool_name == "request_more_time":
2026     game_state.waiting = True
2027     return json.dumps({"acknowledged": True})
2028
2029 else:
2030     return json.dumps({"error": f"Unknown tool: {tool_name}"})
2031
2032 def process_llm_response(message, conversation: list,
2033                         engine: AdaptiveEngine, game_state: GameState,
2034                         preferred_game: Optional[GameType],
2035                         dashboard=None) -> str:
2036     "Handle the LLM response, including any tool call chains."
2037     msg_dict = {"role": "assistant", "content": message.content or ""}
2038     if message.tool_calls: # if tool calls are required inject the function-call into convo
2039         ↪ history so LLM can decide persistence
2040         msg_dict["tool_calls"] = [
2041             {
2042                 "id": tc.id,
2043                 "type": "function",
2044                 "function": {"name": tc.function.name, "arguments": tc.function.arguments},
2045             }
2046             for tc in message.tool_calls # iterate over each tool call in the message
2047         ]

```

```

2047 conversation.append(msg_dict)
2048 # Cap recursive tool calls at 5 rounds to prevent infinite loops
2049 max_tool_rounds = 5
2050 current_msg = message
2051 used_answer_check = False # gate same-chain regen so engine.decide can adapt difficulty first
2052 for _ in range(max_tool_rounds):
2053     if not current_msg.tool_calls:
2054         break # kill loop if no more tool calls
2055
2056     # block same-batch regen; engine.decide() must update difficulty first
2057     batch_has_check = any(tc.function.name == "check_game_answer"
2058                           for tc in current_msg.tool_calls)
2059     for tc in current_msg.tool_calls:
2060         fn_name = tc.function.name
2061         try:
2062             fn_args = json.loads(tc.function.arguments)
2063         except json.JSONDecodeError:
2064             fn_args = {}
2065
2066         if batch_has_check and fn_name == "generate_game_question":
2067             result_str = json.dumps({"skipped": True,
2068                                     "reason": "deferred until after adaptive decision"})
2069             print(f" Skipping {fn_name}: deferred after answer check")
2070         else:
2071             print(f" Calling tool {fn_name}({fn_args})")
2072             result_str = execute_tool_call(
2073                 fn_name, fn_args, engine, game_state,
2074                 conversation, preferred_game, dashboard
2075             )
2076             print(f" Tool returned: {result_str[:120]}")
2077             if fn_name == "check_game_answer":
2078                 used_answer_check = True
2079             conversation.append({
2080                 "role": "tool",
2081                 "tool_call_id": tc.id,
2082                 "content": result_str,
2083             })
2084     # withhold generate_game_question after a check; defer to next turn so engine.decide can
2085     #   ↳ update difficulty first
2086     next_tools = TOOLS
2087     current_msg = converse(conversation, next_tools)
2088     resp_dict = {"role": "assistant", "content": current_msg.content or ""}
2089     if current_msg.tool_calls:
2090         resp_dict["tool_calls"] = [
2091             {
2092                 "id": tc.id,
2093                 "type": "function",
2094                 "function": {"name": tc.function.name, "arguments": tc.function.arguments},
2095             }
2096             for tc in current_msg.tool_calls
2097         ]
2098     conversation.append(resp_dict)
2099
2100     return current_msg.content or ""
2101
2102 #look for gestures
2103 def extract_gesture(text: str) -> str:
2104     "Returns wave always -only gesture now used."
2105     return "wave"

```

```

2105 def validate_numbers_round(numbers: list, target: int) -> bool:
2106     """Skipped -too slow for real-time use. gpt is instructed to verify its own answer."""
2107     return True
2108
2109
2110 def generate_game_question_internal(game_type_str: str, difficulty_str: str,
2111                                   recent: Optional[list[str]] = None,
2112                                   recent_answers: Optional[list[str]] = None,
2113                                   recent_game_types: Optional[list[str]] = None) -> dict:
2114     try:
2115         gt = GameType(game_type_str)
2116     except ValueError:
2117         gt = GameType.NUMBERS
2118     try:
2119         diff = Difficulty[difficulty_str]
2120     except (KeyError, ValueError):
2121         diff = Difficulty.MEDIUM
2122
2123     import secrets
2124     variety_seed = secrets.token_hex(3)
2125
2126     banned_questions = "\n".join(f"- {q}" for q in (recent or [])[-30:])
2127     banned_answers = ", ".join((recent_answers or [])[-30:])
2128
2129     prompt = f"""You are generating a {game_type_str.upper()} round for a Countdown-style game.
2130
2131     GAME TYPE: {game_type_str.upper()} -do not generate any other type.
2132     DIFFICULTY: {DIFFICULTY_DESCRIPTIONS[diff]}
2133     VARIETY SEED (use this to pick different numbers/letters every time): {variety_seed}
2134
2135     STRICT RULES -you MUST follow all of these:
2136     1. This MUST be a {game_type_str.upper()} round. No exceptions.
2137     2. The question MUST be completely different from every question in the BANNED LIST below.
2138     3. The correct answer MUST NOT appear in the BANNED ANSWERS list below.
2139     4. Pick a fresh number set or letter set -do not reuse combinations you have used before.
2140
2141     BANNED LIST (do not repeat or closely mimic any of these):
2142     {banned_questions if banned_questions else "none yet"}
2143
2144     BANNED ANSWERS (your answer must not be any of these):
2145     {banned_answers if banned_answers else "none yet"}
2146
2147     Respond with a JSON object only -no markdown, no code fences:
2148     {"question": "...", "answer": "...", "category": "..."}
2149 """
2150
2151     try:
2152         resp = client.chat.completions.create(
2153             model="gpt-5.4",
2154             messages=[
2155                 {"role": "system", "content": (
2156                     "You generate countdown-style game questions. Respond ONLY with valid JSON. "
2157                     "Follow all rules in the user prompt exactly. "
2158                     "For numbers rounds you MUST verify your arithmetic before responding. "
2159                     "The 'answer' field must contain ONLY the final solution expression -"
2160                     "for example '50 + 25 = 75'. "
2161                     "Do NOT include working out, alternative attempts, explanations, "
2162                     "or commentary in the answer field. One clean expression only."
2163                 )},

```

```

2164         {"role": "user", "content": prompt},
2165     ],
2166     temperature=1.2,
2167     timeout=API_TIMEOUT,
2168 )
2169 content = resp.choices[0].message.content.strip()
2170 if content.startswith("`"):
2171     content = content.split("\n", 1)[1] if "\n" in content else content[3:]
2172     if content.endswith("`"):
2173         content = content[:-3]
2174     content = content.strip()
2175 result = json.loads(content)
2176
2177 # validate numbers rounds as gpt normally hallucinates unreachable targets
2178 if game_type_str == "numbers":
2179     try:
2180         import re
2181         q = result.get("question", "")
2182         nums = list(map(int, re.findall(r'\b\d+\b', q.split("make")[0])))
2183         target_match = re.search(r'make.*?(\d+)', q)
2184         if target_match and nums:
2185             target = int(target_match.group(1))
2186             if not validate_numbers_round(nums, target):
2187                 print(f"gpt produced unreachable target {target} from {nums}; using fallback")
2188                 return {
2189                     "question": "Using 25, 50, 3, 6, 4, and 2, make the target 100.",
2190                     "answer": "25 x 4 = 100",
2191                     "category": "numbers fallback",
2192                 }
2193         except Exception as ve:
2194             print(f" Numbers validation skipped: {ve}")
2195     return result
2196 except json.JSONDecodeError:
2197     return {"question": content if 'content' in locals() else "", "answer": "", "category":
2198         ↪ "general"}
2199 except Exception as e:
2200     print(f" Game question gen failed: {e}")
2201     return {
2202         "question": "Let's try a quick one: what is 25 + 17?",
2203         "answer": "42",
2204         "category": "arithmetic fallback",
2205     }
2206
2207 _STATE_COLOURS = {
2208     InferredState.THRIVING: "green",
2209     InferredState.COMFORTABLE: "blue",
2210     InferredState.STRUGGLING: "orange",
2211     InferredState.FRUSTATED: "red",
2212     InferredState.DISENGAGED: "grey",
2213 }
2214
2215 class GazeDashboard:
2216     """Tkinter dashboard for GAZE.
2217     1- create a window with two panels: left = conversation then video, right for inferred
2218         ↪ stats/signals/state
2219     2- left panel: camera canvas above the scrollable conversation log; log is read-only default
2220         ↪ and

```

```

2220         flipped to "normal" when a new line is appended
2221     3- right panel: round/score/streak counters at top, then the transcription block (what the
        ↳ user said + correct/incorrect,
2222         recoloured per outcome), then the live signals (face CNN, voice MLP, volume RMS, response
        ↳ time, rolling accuracy,
2223         think budget), then the adaptive decision block (inferred state, difficulty, tone,
        ↳ adaptations, plus a grey eval-note line below)
2224     4- quit handling: the Quit button, Esc, Cmd/Ctrl-Q and the window-close (X) all converge on
        ↳ quit_app(), so the SSH farewell + LED-off
2225         always run no matter how the user closes the dashboard
2226     5- refresh loops: camera_refresh() composites the latest raw frame with the cached detection
        ↳ overlay at native fps; signal_refresh()
2227         repaints the StringVar-bound labels at a slower cadence; both self-reschedule via
        ↳ root.after()
2228 ""
2229
2230 CAMERA_W, CAMERA_H = 400, 300
2231
2232 def __init__(self, ssh=None, ssh_tts=None):
2233     # stashed for quit_app farewell + LED-off
2234     self._ssh = ssh
2235     self._ssh_tts = ssh_tts
2236     self.root = tk.Tk()
2237     self.root.title("GAZE Dashboard")
2238     self.root.resizable(False, False)
2239
2240     # left col: video + chat; right col: stats
2241     left = tk.Frame(self.root)
2242     left.grid(row=0, column=0, padx=8, pady=8, sticky="n")
2243
2244     # camera canvas; black letterboxes empty pixels
2245     self.camera_label = tk.Label(left, bg="black",
2246                                 width=self.CAMERA_W, height=self.CAMERA_H)
2247     self.camera_label.pack()
2248
2249     tk.Label(left, text="Conversation:").pack(anchor="w", pady=(6, 0))
2250     conv_frame = tk.Frame(left)
2251     conv_frame.pack()
2252     # read-only by default; helpers flip to insert
2253     self._conv_text = tk.Text(conv_frame, font=("Courier", 10),
2254                               width=50, height=10, wrap="word",
2255                               state="disabled")
2256     # scrollbar bidirectional wiring
2257     conv_scroll = tk.Scrollbar(conv_frame, command=self._conv_text.yview)
2258     self._conv_text.configure(yscrollcommand=conv_scroll.set)
2259     self._conv_text.pack(side="left", fill="both", expand=True)
2260     conv_scroll.pack(side="right", fill="y")
2261
2262     right = tk.Frame(self.root)
2263     right.grid(row=0, column=1, padx=(0, 8), pady=8, sticky="n")
2264
2265     tk.Label(right, text="GAZE (Game-Adaptive Zone of Engagement) Dashboard",
2266             font=("TkDefaultFont", 14, "bold")).pack(pady=(0, 4))
2267
2268     self._round_var = tk.StringVar(value="Round: -")
2269     self._score_var = tk.StringVar(value="Score: 0/0")
2270     self._streak_var = tk.StringVar(value="Streak: 0")
2271     for var in (self._round_var, self._score_var, self._streak_var):
2272         tk.Label(right, textvariable=var).pack(anchor="w")

```

```

2273
2274 # user transcription only; answer on stdout
2275 tk.Label(right, text="").pack()
2276 tk.Label(right, text="Transcription:",
2277          font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2278 self._heard_var = tk.StringVar(value="You said: -")
2279 self._result_var = tk.StringVar(value="")
2280 tk.Label(right, textvariable=self._heard_var).pack(anchor="w")
2281 # fg recolour on correctness
2282 self._result_label = tk.Label(right, textvariable=self._result_var,
2283                               font=("TkDefaultFont", 11, "bold"))
2284 self._result_label.pack(anchor="w")
2285
2286 tk.Label(right, text="").pack()
2287 tk.Label(right, text="Live Signals:",
2288          font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2289 self._face_var = tk.StringVar(value="Face (CNN): -")
2290 self._voice_var = tk.StringVar(value="Voice (MLP): -")
2291 self._vol_var = tk.StringVar(value="Volume RMS: -")
2292 self._time_var = tk.StringVar(value="Response time: -")
2293 self._acc_var = tk.StringVar(value="Rolling acc: -")
2294 self._budget_var = tk.StringVar(value="Think budget: -")
2295 for var in (self._face_var, self._voice_var, self._vol_var,
2296            self._time_var, self._acc_var, self._budget_var):
2297     tk.Label(right, textvariable=var,
2298             font=("Courier", 10)).pack(anchor="w")
2299
2300 tk.Label(right, text="").pack()
2301 tk.Label(right, text="Adaptive Decision:",
2302          font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2303 self._state_var = tk.StringVar(value="State: -")
2304 # kept for state-coloured fg
2305 self._state_label = tk.Label(right, textvariable=self._state_var,
2306                              font=("TkDefaultFont", 11, "bold"))
2307 self._state_label.pack(anchor="w")
2308
2309 self._diff_var = tk.StringVar(value="Difficulty: -")
2310 self._tone_var = tk.StringVar(value="Tone: -")
2311 self._adapt_var = tk.StringVar(value="Adaptations: -")
2312 for var in (self._diff_var, self._tone_var, self._adapt_var):
2313     tk.Label(right, textvariable=var).pack(anchor="w")
2314
2315 # adaptation-eval note (miniscule 'grey' text)
2316 self._eval_var = tk.StringVar(value="")
2317 self._eval_label = tk.Label(right, textvariable=self._eval_var,
2318                             font=("TkDefaultFont", 9), fg="grey",
2319                             wraplength=340, justify="left")
2320 self._eval_label.pack(anchor="w")
2321
2322 tk.Button(right, text="Quit (Esc)",
2323          command=self.quit_app).pack(pady=(10, 0))
2324
2325 self.root.bind("<Escape>", lambda e: self.quit_app())
2326 # Cmd-Q is macOS-only; guard each
2327 for combo in ("<Command-q>", "<Control-q>"):
2328     try:
2329         self.root.bind(combo, lambda e: self.quit_app())
2330     except tk.TclError:
2331         pass

```

```

2332     # route window-close through quit_app
2333     self.root.protocol("WM_DELETE_WINDOW", self.quit_app)
2334
2335     self.camera_refresh()
2336     self.signal_refresh()
2337     self.root.update()                                # paint before mainloop()
2338
2339
2340 def camera_refresh(self):
2341     """Composite the latest raw frame with the cached detection overlay.
2342     Bbox is drawn on every fresh raw frame for a live-feed."""
2343     with _preview_lock:
2344         frame = _preview_frame
2345         bbox = _last_bbox
2346         emotion = _preview_state["emotion"]
2347         conf = _preview_state["confidence"]
2348
2349     if frame is not None:
2350         display = frame.copy()                                # don't mutate the shared raw frame; the
                ↳ bbox/text overlays are per-paint
2351         if bbox is not None:
2352             x, y, w, h = bbox
2353             cv2.rectangle(display, (x, y), (x + w, y + h), (0, 255, 0), 2)
2354             label = f"{emotion} ({conf:.0%})"
2355             cv2.putText(display, label, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
2356                        0.8, (0, 255, 0), 2)
2357             rgb = cv2.cvtColor(display, cv2.COLOR_BGR2RGB) # OpenCV is BGR, Tk/PIL expect RGB; swap
                ↳ channels here
2358             small = cv2.resize(rgb, (self.CAMERA_W, self.CAMERA_H))
2359             img = ImageTk.PhotoImage(Image.fromarray(small)) # Tk-compatible image wrapper around
                ↳ the PIL image
2360             self.camera_label.configure(image=img)
2361             self.camera_label._photo = img # prevent garbage collection -Tk doesn't keep a ref to
                ↳ PhotoImage so without an attribute hold the image gets GC'd and the canvas blanks
2362
2363     self.root.after(33, self.camera_refresh) # ~30 fps target -Tk's standard self-rescheduling
                ↳ animation pattern; after(ms, fn) queues fn onto the mainloop after ms milliseconds
2364
2365 def signal_refresh(self):
2366     with _preview_lock:
2367         emotion = _preview_state["emotion"]
2368         conf = _preview_state["confidence"]
2369     self._face_var.set(f"Face (CNN): {emotion} ({conf:.0%})")
2370     self.root.after(500, self.signal_refresh) # 2 Hz
2371
2372 # thread-safe scheduling: tkinter widgets are main-thread only (macOS); root.after(0, fn)
                ↳ queues fn onto the main loop.
2373
2374 def on_main(self, fn):
2375     "Schedule fn to run on the main (tkinter) thread."
2376     try:
2377         self.root.after(0, fn)
2378     except tk.TclError:
2379         pass
2380
2381
2382 def update_robot_speech(self, text: str):
2383     # flip read-only widget to insert, scroll, re-disable
2384     def apply():

```

```

2385         self._conv_text.configure(state="normal")
2386         self._conv_text.insert("end", f"Robot: {text}\n\n")
2387         self._conv_text.see("end")
2388         self._conv_text.configure(state="disabled")
2389     self.on_main(apply) # schedule the UI update on the main thread
2390
2391 def append_user_speech(self, text: str):
2392     def apply():
2393         self._conv_text.configure(state="normal")
2394         self._conv_text.insert("end", f"You: {text}\n")
2395         self._conv_text.see("end")
2396         self._conv_text.configure(state="disabled")
2397         self._heard_var.set(f"You said: {text}") # mirror to right panel
2398     self.on_main(apply)
2399
2400 def update_think_budget(self, secs: float):
2401     "Update the dashboard's adaptive think-budget diagnostic row."
2402     def apply():
2403         self._budget_var.set(f"Think budget: {secs:.1f}s")
2404     self.on_main(apply)
2405
2406 def update_signals(self, round_num: int, user_answer: str, correct_answer: str,
2407                   correct: bool, expression: str, expr_conf: float,
2408                   vocal_emo: str, vocal_conf: float, vol_rms: float,
2409                   response_time: float, rolling_acc: float,
2410                   total_correct: int, total_rounds: int, streak: int):
2411     def apply():
2412         self._round_var.set(f"Round: {round_num}")
2413         self._score_var.set(f"Score: {total_correct}/{total_rounds}")
2414         self._streak_var.set(f"Streak: {streak}")
2415
2416         self._heard_var.set(f"You said: {user_answer if user_answer else '(silence)'}") #
2417             ↳ extract user's answer
2418         if not correct_answer: # baseline; non-active game state
2419             self._result_var.set("")
2420             self._result_label.configure(fg="grey")
2421         elif correct:
2422             self._result_var.set("CORRECT")
2423             self._result_label.configure(fg="green")
2424         elif not user_answer:
2425             self._result_var.set("NO ANSWER")
2426             self._result_label.configure(fg="grey")
2427         else:
2428             self._result_var.set("INCORRECT")
2429             self._result_label.configure(fg="red")
2430
2431         self._face_var.set(f"Face (CNN): {expression} ({expr_conf:.0%})")
2432         self._voice_var.set(f"Voice (MLP): {vocal_emo} ({vocal_conf:.0%})")
2433
2434         vol_tag = "(loud)" if vol_rms > 2000 else "(quiet)" if vol_rms < VOLUME_THRESHOLD else
2435             ↳ "(normal)"
2436         bar_len = min(int(vol_rms / 250), 20)
2437         meter = "#" * bar_len + "." * (20 - bar_len)
2438         self._vol_var.set(f"Volume RMS: {vol_rms:>5.0f} [{meter}] {vol_tag}")
2439         self._time_var.set(f"Response time: {response_time:.1f}s")
2440         self._acc_var.set(f"Rolling acc: {rolling_acc:.0%}")
2441
2442     self.on_main(apply)

```



```

2442 def update_decision(self, decision, adaptation_eval: str = None):
2443     state = decision.inferred_state
2444     def apply():
2445         self._state_var.set(f"State: {state.value.upper()}")
2446         self._state_label.configure(fg=_STATE_COLOURS.get(state, "grey"))
2447         self._diff_var.set(f"Difficulty: {decision.difficulty.name}")
2448         self._tone_var.set(f"Tone: {decision.tone}")
2449         flags = []
2450         if decision.give_hint: flags.append("hint")
2451         if decision.give_encouragement: flags.append("encouragement")
2452         if decision.switch_game: flags.append(f"switch -> {decision.game_type.value}")
2453         self._adapt_var.set(f"Adaptations: {'', '.join(flags) if flags else 'none'}")
2454         self._eval_var.set(adaptation_eval if adaptation_eval else "")
2455     self.on_main(apply)
2456
2457 def quit_app(self):
2458     "Speak a farewell, dim Pepper's eye-LEDs, then kill the process."
2459     print("\n [Dashboard] Quit requested.")
2460     if not LOCAL_MODE and self._ssh is not None:
2461         nao_set_leds(self._ssh, "FaceLeds", 0x00000000, 0.5)
2462         # local_say in LOCAL_MODE; SSH-TTS otherwise
2463         say(self._ssh_tts, "It was great to chat! See you next time!")
2464     self.close()
2465     os._exit(0)
2466
2467 def close(self):
2468     try:
2469         self.root.destroy()
2470     except tk.TclError:
2471         pass
2472
2473 def is_goodbye(text: str) -> bool:
2474     "LLM goodbye-intent gate; True if the user signalled they want to leave."
2475     if not text:
2476         return False
2477     try:
2478         bye = client.chat.completions.create(
2479             model="gpt-4.1", max_completion_tokens=5, temperature=0.0,
2480             timeout=API_TIMEOUT,
2481             messages=[{"role": "system", "content":
2482                 "Did the user just signal they want to end the conversation (e.g. 'bye', 'goodbye',
2483                 ↪ 'I'm done', 'see you later', 'I have to go')? Reply ONLY with 'GOODBYE' or
2484                 ↪ 'CONTINUE'."},
2485                 {"role": "user", "content": text}],
2486             ).choices[0].message.content.strip().upper()
2487     except Exception:
2488         return False
2489     return "GOODBYE" in bye
2490
2491 def conversation_loop(dashboard, face_model, face_cascade, speech_model,
2492     local_camera, ssh, ssh_tts, energy_threshold):
2493     "Comprehensive main-conversation loop."
2494     preferred_game = None
2495     engine = AdaptiveEngine()
2496     game_state = GameState()
2497
2498     save_data = load_session()
2499     if save_data:

```

```

2499     # legacy fallback; older saves only had per-session rounds_played
2500     prev_rounds = save_data.get("total_rounds_played",
2501                                save_data.get("rounds_played", 0))
2502     prev_correct = save_data.get("total_correct", 0)
2503
2504     welcome_back = f"Welcome back! Last time you played {prev_rounds} rounds and got
2505         ↪ {prev_correct} correct. Want to continue where you left off, or start fresh?"
2506
2507     if not LOCAL_MODE:
2508         nao_track_face(ssh, enable=True)
2509         nao_set_leds(ssh, "FaceLeds", 0x0000FF00, 1.0)
2510         nao_gesture(ssh, "wave")
2511         say(ssh_tts, welcome_back)
2512         print(f"\nRobot: {welcome_back}")
2513         dashboard.update_robot_speech(welcome_back)
2514
2515     print("\nListening for continue/fresh...")
2516     if not LOCAL_MODE:
2517         nao_set_leds(ssh, "EarLeds", 0x0000FF00, 0.3)
2518         record(ssh, energy_threshold,
2519                no_speech_max=4.0, silence_secs=2.0, record_max_secs=6.0)
2520
2521     expression, expr_conf = capture_and_classify(
2522         ssh, face_model, face_cascade, local_camera)
2523     vocal_emo, vocal_conf = classify_speech_emotion(speech_model, LOCAL_WAV)
2524     vol_rms = measure_volume()
2525     dashboard.update_signals(
2526         round_num=0, user_answer="", correct_answer="", correct=False,
2527         expression=expression, expr_conf=expr_conf,
2528         vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2529         vol_rms=vol_rms, response_time=0, rolling_acc=0,
2530         total_correct=0, total_rounds=0, streak=0,
2531     )
2532
2533     resume_text = transcribe(
2534         bypass_wake_word=True,
2535         record_again=lambda: record(ssh, energy_threshold,
2536                                    no_speech_max=4.0, silence_secs=2.0, record_max_secs=6.0),
2537     )
2538
2539     if resume_text:
2540         print(f" Heard: {resume_text}")
2541         dashboard.append_user_speech(resume_text)
2542         if is_goodbye(resume_text):
2543             dashboard.quit_app()
2544     else:
2545         print(" Heard: (silence)")
2546
2547     # LLM intent classifier; on failure fall back to a keyword check, then
2548     # default to CONTINUE so an API timeout never silently nukes the save.
2549     intent = ""
2550     if resume_text:
2551         try:
2552             intent = client.chat.completions.create(
2553                 model="gpt-4.1", max_completion_tokens=5, temperature=0.0,
2554                 timeout=API_TIMEOUT,
2555                 messages=[{"role": "system", "content":
2556                     "The user was just asked whether they want to CONTINUE their previous session
2557                         ↪ or start FRESH. Reply ONLY with 'CONTINUE' or 'FRESH'."},
2558                     {"role": "user", "content": resume_text}],
2559                 ).choices[0].message.content.strip().upper()

```

```

2556         except Exception as e:
2557             print(f" Resume intent classification failed ({e}); falling back to keyword match")
2558             low = resume_text.lower()
2559             fresh_kw = ("fresh", "start over", "restart", "new game", "start again", "begin
                ↪ again", "from scratch")
2560             continue_kw = ("continue", "carry on", "keep going", "resume", "where i left",
                ↪ "where we left", "yes", "yeah", "yep")
2561             if any(k in low for k in fresh_kw):
2562                 intent = "FRESH"
2563             elif any(k in low for k in continue_kw):
2564                 intent = "CONTINUE"
2565             else:
2566                 # ambiguous; preserve the save rather than wipe it
2567                 intent = "CONTINUE"
2568             print(f" Keyword fallback resolved intent: {intent}")
2569         if "CONTINUE" in intent:
2570             engine = restore_engine(save_data)
2571             preferred_game = engine.current_game
2572             restored_rounds = save_data.get("total_rounds_played",
2573                 ↪ save_data.get("rounds_played", 0))
2574             restore_msg = f"Restoring your previous session: {restored_rounds} rounds on record."
2575             say(ssh_tts, restore_msg)
2576             print(f"\nRobot: {restore_msg}")
2577         else:
2578             delete_save()
2579             fresh_msg = "No worries, starting fresh! Your previous save has been cleared."
2580             say(ssh_tts, fresh_msg)
2581             print(f"\nRobot: {fresh_msg}")
2582
2583
2584     if not LOCAL_MODE:
2585         nao_track_face(ssh, enable=True)
2586         nao_set_leds(ssh, "FaceLeds", 0x0000FF00, 1.0)
2587
2588     conversation = [{"role": "system", "content": BASE_SYSTEM_PROMPT}]
2589
2590     greeting_directive = "[Give a warm, supportive greeting. Mention you can chat, play games, or
        ↪ just hang out. Keep it to 2 sentences.]"
2591     greeting_msg = converse(
2592         conversation + [{"role": "user", "content": greeting_directive}],
2593         TOOLS,
2594     )
2595     greeting_text = greeting_msg.content or "Hello, lovely to meet you!"
2596     greeting_gesture = extract_gesture(greeting_text)
2597     greeting_speech = re.sub(r'\[gesture:\w+\]', '', greeting_text).strip()
2598
2599     conversation.append({"role": "assistant", "content": greeting_text})
2600
2601     if not LOCAL_MODE:
2602         nao_gesture(ssh, "wave")
2603         say(ssh_tts, greeting_speech)
2604         print(f"\nRobot: {greeting_speech}")
2605         dashboard.update_robot_speech(greeting_speech)
2606
2607
2608     turn_count = 0
2609     # tiered disengagement; check-in at 3, game-switch at 4
2610     nudge_level = 0
2611

```

```

2612     try:
2613         while True:
2614             turn_count += 1
2615             print(f"\n{'-' * 40} Turn {turn_count} {'-' * 40}")
2616
2617             print("Capturing expression...")
2618             expression, expr_conf = capture_and_classify(
2619                 ssh, face_model, face_cascade, local_camera
2620             )
2621             if expr_conf < FACE_CONFIDENCE_THRESHOLD:
2622                 print(f" Expression: {expression} ({expr_conf:.2f}): LOW CONFIDENCE, treating as
                ↪ Neutral")
2623                 expression = "Neutral"
2624             else:
2625                 print(f" Expression: {expression} ({expr_conf:.2f})")
2626
2627             question_start = time.time()
2628             print("Listening...")
2629             if not LOCAL_MODE:
2630                 # brief cyan pulse on the face LEDs so the user can see the robot is actively
                ↪ listening; ears also go green
2631                 nao_set_leds(ssh, "FaceLeds", 0x0000FFFF, 0.2)
2632                 nao_set_leds(ssh, "EarLeds", 0x0000FF00, 0.3)
2633
2634                 # adaptive think-budget based on previous round's state + current face
2635                 prev_state = (engine.history[-1].inferred_state if engine.history
2636                             else InferredState.COMFORTABLE)
2637                 prev_rt = engine.history[-1].response_time if engine.history else 0.0
2638                 no_speech_max, silence_secs, record_max_secs = engine.recommend_think_budget(
2639                     state=prev_state, expression=expression,
2640                     prev_response_time=prev_rt,
2641                     consecutive_silences=engine.consecutive_silences,
2642                     waiting=game_state.waiting,
2643                 )
2644                 dashboard.update_think_budget(record_max_secs)
2645
2646                 record(ssh, energy_threshold,
2647                        no_speech_max=no_speech_max,
2648                        silence_secs=silence_secs,
2649                        record_max_secs=record_max_secs)
2650                 response_time = time.time() - question_start
2651
2652                 # extended think-budget consumed; reset for next turn
2653                 if game_state.waiting:
2654                     game_state.waiting = False
2655
2656                 vocal_emo, vocal_conf = classify_speech_emotion(speech_model, LOCAL_WAV)
2657                 if vocal_conf < VOICE_CONFIDENCE_THRESHOLD:
2658                     print(f" Vocal emotion: {vocal_emo} ({vocal_conf:.2f}): LOW CONFIDENCE, treating as
                    ↪ neutral")
2659                     vocal_emo = "neutral"
2660                 else:
2661                     print(f" Vocal emotion: {vocal_emo} ({vocal_conf:.2f})")
2662
2663                 vol_rms = measure_volume()
2664                 print(f" Volume RMS: {vol_rms:.0f}")
2665
2666                 # (c) transcribe; per-chunk gate, file-wide RMS removed
2667                 relisten = lambda: record(ssh, energy_threshold,

```

```

2668             no_speech_max=no_speech_max,
2669             silence_secs=silence_secs,
2670             record_max_secs=record_max_secs)
2671 if INPUT_IS_LOCAL: # hybrid check
2672     if _local_speech_detected:
2673         user_text = transcribe(bypass_wake_word=True, record_again=relisten)
2674     else:
2675         print(f" No speech chunk detected (floor={LOCAL_SILENCE_RMS}); skipping
                ↪ transcription.")
                user_text = ""
2676 elif vol_rms >= NAO_MIN_RMS_TO_TRANSCRIBE:
2677     print(f" NAO RMS diagnostic: {vol_rms:.0f} (gate floor {NAO_MIN_RMS_TO_TRANSCRIBE})")
2678     user_text = transcribe(bypass_wake_word=True, record_again=relisten)
2679 else:
2680     print(f" WAV near-empty ({vol_rms:.0f} < {NAO_MIN_RMS_TO_TRANSCRIBE}); skipping
                ↪ transcription.")
2681     user_text = ""
2682
2683
2684 if user_text:
2685     print(f" Heard: {user_text}")
2686     dashboard.append_user_speech(user_text)
2687     # user spoke; reset silence + nudge so future silence re-arms from scratch
2688     engine.consecutive_silences = 0
2689     nudge_level = 0
2690
2691     if is_goodbye(user_text):
2692         dashboard.quit_app()
2693 else:
2694     print(" Heard: (silence)")
2695     dashboard.append_user_speech("(silence)")
2696     engine.consecutive_silences += 1
2697
2698     # surface signals to dashboard even though the LLM is skipped this turn
2699     dashboard.update_signals(
2700         round_num=turn_count, user_answer="", correct_answer="",
2701         correct=False, expression=expression, expr_conf=expr_conf,
2702         vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2703         vol_rms=vol_rms, response_time=response_time,
2704         rolling_acc=engine.rolling_correctness(),
2705         total_correct=engine.total_correct,
2706         total_rounds=engine.total_rounds_played,
2707         streak=engine.consecutive_correct,
2708     )
2709
2710     # 1- check-in at 3 silences (proposal: "intervenes to re-engage them")
2711     if engine.consecutive_silences >= 3 and nudge_level < 1:
2712         nudge_prompt = [
2713             {"role": "system",
2714              "content": BASE_SYSTEM_PROMPT},
2715             {"role": "user",
2716              "content": "[The user has been quiet for 3 turns. Offer ONE brief, gentle
                ↪ check-in; no question, no nagging. 1 sentence max.]"},
2717         ]
2718         nudge_msg = converse(nudge_prompt, [])
2719         nudge_text = (nudge_msg.content or "").strip()
2720         nudge_speech = re.sub(r'\[gesture:\w+\]', '', nudge_text).strip()
2721         if not LOCAL_MODE:
2722             # recolour eyes to signal the state has shifted; user sees the shift even if
                ↪ they say nothing back

```

```

2723         nao_set_leds(ssh, "FaceLeds",
2724                     LED_COLOURS[InferredState.DISENGAGED], 0.4)
2725     if nudge_speech:
2726         say(ssh_tts, nudge_speech)
2727         print(f"\nRobot (nudge): {nudge_speech}")
2728         dashboard.update_robot_speech(nudge_speech)
2729         nudge_level = 1
2730
2731     # 2- switch game, soft re-invitation
2732     elif engine.consecutive_silences >= 4 and nudge_level < 2:
2733         engine.current_game = engine.pick_different_game()
2734         engine.game_switch_count += 1
2735         escalate_prompt = [
2736             {"role": "system",
2737              "content": BASE_SYSTEM_PROMPT},
2738             {"role": "user",
2739              "content": f"[The user has gone silent for 4 turns. Warmly acknowledge
2740                        ↪ they've gone quiet then offer a {engine.current_game.value} round as a
2741                        ↪ soft alternative. Two sentences total.]"},
2742             ]
2743         esc_msg = converse(escalate_prompt, [])
2744         esc_text = (esc_msg.content or "").strip()
2745         esc_speech = re.sub(r'\[gesture:\w+\]', '', esc_text).strip()
2746         if not LOCAL_MODE:
2747             nao_set_leds(ssh, "FaceLeds",
2748                         LED_COLOURS[InferredState.DISENGAGED], 0.4)
2749         if esc_speech:
2750             say(ssh_tts, esc_speech)
2751             print(f"\nRobot (escalate): {esc_speech}")
2752             dashboard.update_robot_speech(esc_speech)
2753             nudge_level = 2
2754
2755         continue # skip the LLM call; just wait for the user
2756
2757     if user_text.lower().strip() in [
2758         "stop", "quit", "exit", "goodbye", "bye", "end",
2759         "i want to stop", "let's stop", "no more",
2760     ]:
2761         print("Patient wants to stop.")
2762         break
2763
2764     signal_ctx = build_signal_context(
2765         engine, expression, expr_conf,
2766         vocal_emo, vocal_conf, vol_rms, response_time,
2767     )
2768
2769     user_msg_content = f"{signal_ctx}\n\nUser says: {user_text}"
2770     conversation.append({"role": "user", "content": user_msg_content})
2771
2772     # trim conversation to prevent context overflow; keep system + last 40 messages (20
2773         ↪ exchanges)
2774     if len(conversation) > 42:
2775         conversation = [conversation[0]] + conversation[-40:]
2776
2777     if not LOCAL_MODE:
2778         nao_set_leds(ssh, "EarLeds", 0x000000FF, 0.3) # blue = thinking
2779
2780     llm_message = converse(conversation, TOOLS)
2781

```

```

2779 response_text = process_llm_response(
2780     llm_message, conversation, engine, game_state,
2781     preferred_game, dashboard
2782 )
2783
2784 if not response_text.strip():
2785     response_text = "I'm here! What would you like to talk about? [gesture:neutral]"
2786
2787 gesture_type = extract_gesture(response_text)
2788 speech_text = re.sub(r'\[gesture:\w+\]', '', response_text).strip()
2789
2790 if not LOCAL_MODE:
2791     if engine.adaptation_log:
2792         last_state_str = engine.adaptation_log[-1]["state"]
2793         try:
2794             last_state = InferredState(last_state_str)
2795         except ValueError:
2796             last_state = InferredState.COMFORTABLE
2797         nao_set_leds(
2798             ssh, "FaceLeds",
2799             LED_COLOURS.get(last_state, 0x00FFFFFF), 0.5
2800         )
2801
2802 say(ssh_tts, speech_text)
2803
2804 print(f"\nRobot: {speech_text}")
2805 if game_state.current_answer:
2806     print(f"(Game answer: {game_state.current_answer})")
2807 dashboard.update_robot_speech(speech_text)
2808
2809 # (k) update dashboard (signals + decision if game active); use the actual answer check
2810 ↳ result from the tool chain
2811 was_game_answer = game_state.last_answer_checked
2812 correct = game_state.last_answer_correct if was_game_answer else False
2813
2814 if not LOCAL_MODE and was_game_answer and correct:
2815     threading.Thread(target=nao_gesture, args=(ssh, "correct_wave"), daemon=True).start()
2816
2817 # only run engine.decide on real game answers; chat or more-time turns mustn't ramp
2818 ↳ difficulty or pollute streak counters
2819 if was_game_answer:
2820     # use snapshot so a same-chain generate_game_question can't pollute this round's
2821     ↳ logged values
2822     answered_q = game_state.answered_question or game_state.current_question
2823     answered_a = game_state.answered_answer or game_state.current_answer
2824     answered_gt = game_state.answered_game_type or engine.current_game
2825     answered_df = game_state.answered_difficulty or engine.current_difficulty
2826     decision = engine.decide(
2827         expression, expr_conf, response_time,
2828         correct=correct,
2829         answer_text=user_text,
2830         vocal_emotion=vocal_emo, vocal_conf=vocal_conf,
2831         volume_rms=vol_rms,
2832     )
2833     engine.record_round(RoundResult(
2834         round_number=turn_count,
2835         game_type=answered_gt,
2836         difficulty=answered_df,
2837         question=answered_q,

```

```

2835         user_answer=user_text,
2836         correct=correct,
2837         response_time=response_time,
2838         facial_expression=expression,
2839         expression_confidence=expr_conf,
2840         vocal_emotion=vocal_emo,
2841         vocal_emotion_confidence=vocal_conf,
2842         volume_rms=vol_rms,
2843         inferred_state=decision.inferred_state,
2844     ))
2845     # save after every round; power-cycle loses one turn, not five
2846     save_session(engine, preferred_game, quiet=True)
2847     dashboard.update_signals(
2848         round_num=turn_count, user_answer=user_text,
2849         correct_answer=answered_a,
2850         correct=correct,
2851         expression=expression, expr_conf=expr_conf,
2852         vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2853         vol_rms=vol_rms, response_time=response_time,
2854         rolling_acc=engine.rolling_correctness(),
2855         total_correct=engine.total_correct,
2856         total_rounds=engine.total_rounds_played,
2857         streak=engine.consecutive_correct,
2858     )
2859     dashboard.update_decision(decision)
2860     # clear snapshot now the round is logged
2861     game_state.answered_question = ""
2862     game_state.answered_answer = ""
2863     game_state.answered_game_type = None
2864     game_state.answered_difficulty = None
2865     else:
2866         # chat or more-time turn; refresh dashboard signals only
2867         dashboard.update_signals(
2868             round_num=turn_count, user_answer=user_text,
2869             correct_answer="",
2870             correct=False,
2871             expression=expression, expr_conf=expr_conf,
2872             vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2873             vol_rms=vol_rms, response_time=response_time,
2874             rolling_acc=engine.rolling_correctness(),
2875             total_correct=engine.total_correct,
2876             total_rounds=engine.total_rounds_played,
2877             streak=engine.consecutive_correct,
2878         )
2879
2880     game_state.last_answer_checked = False
2881
2882     # auto-save every 2 turns
2883     if turn_count % 2 == 0:
2884         save_session(engine, preferred_game, quiet=True)
2885         print(" [Auto-save]")
2886
2887     except KeyboardInterrupt:
2888         print("\n\nInterrupted.")
2889
2890
2891     save_session(engine, preferred_game)
2892
2893     summary = engine.get_session_summary()

```



```

2894     print(f"\n{'=' * 60}")
2895     print(" SESSION SUMMARY")
2896     print(f"{'=' * 60}")
2897     for k, v in summary.items():
2898         print(f" {k}: {v}")
2899
2900     if summary.get("rounds", 0) > 0:
2901         acc = summary["accuracy"]
2902         streak_note = (f" Your best streak was {summary['best_streak']} in a row!"
2903                        if summary["best_streak"] >= 3 else "")
2904         if acc >= 0.8:
2905             farewell = f"Amazing session! You got {summary['correct']} out of {summary['rounds']}
2906                        ↪ right.{streak_note} Brilliant work! Your progress is saved; see you next time!"
2907         elif acc >= 0.5:
2908             farewell = f"Great effort! You scored {summary['correct']} out of
2909                        ↪ {summary['rounds']}.{streak_note} Well played! Your progress is saved; see you
2910                        ↪ next time!"
2911         else:
2912             farewell = f"Thanks for playing! You got {summary['correct']} out of
2913                        ↪ {summary['rounds']}.{streak_note} Every round is a learning opportunity. Your
2914                        ↪ progress is saved; see you next time!"
2915     else:
2916         farewell = "It was great to chat! Your progress is saved; see you next time!"
2917
2918     if not LOCAL_MODE:
2919         nao_gesture(ssh, "wave")
2920     say(ssh_tts, farewell)
2921     print(f"\nRobot: {farewell}")
2922
2923     if local_camera is not None:
2924         local_camera.release()
2925     if not LOCAL_MODE:
2926         nao_track_face(ssh, enable=False)
2927         nao_set_leds(ssh, "FaceLeds", 0x00000000, 0.5)
2928         ssh.close()
2929         ssh_tts.close()
2930     dashboard.close()
2931     print("\nGAZE disconnected.")
2932
2933     # ----- MAIN ENTRY-POINT -----
2934
2935     def main():
2936         print("-----")
2937         print(" GAZE: Game-Adaptive Zone of Engagement")
2938         print(" Adaptive Game-System Ran on Pepper")
2939         print("-----")
2940
2941         print("\nLoading facial expression model...")
2942         face_model = FacialExpressionModel(MODEL_JSON, MODEL_WEIGHTS)
2943         face_cascade = cv2.CascadeClassifier(HAAR_CASCADE)
2944         print(" Facial model loaded.")
2945
2946         speech_model = None
2947         if os.path.exists(SPEECH_MODEL):
2948             print("Loading speech emotion model...")
2949             speech_model = SpeechEmotionModel(SPEECH_MODEL)
2950             print(" Speech model loaded.")
2951         else:
2952             print(f" Speech emotion model not found at {SPEECH_MODEL}; vocal signal disabled.")

```

```

2948         print(" Run train_speech_model.py to generate it.")
2949
2950     local_camera = None
2951     if USE_LOCAL_CAMERA:
2952         local_camera = cv2.VideoCapture(0)
2953         # BUFFER_SIZE=1; else OpenCV buffers stale frames
2954         local_camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
2955         local_camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
2956         local_camera.set(cv2.CAP_PROP_FPS, 30)
2957         local_camera.set(cv2.CAP_PROP_BUFFERSIZE, 1)
2958         print(" Using local webcam for expression detection.")
2959         if DEBUG_PREVIEW:
2960             start_preview_thread(local_camera, face_model, face_cascade)
2961             print(" Preview thread started.")
2962     else:
2963         print(" Using Pepper's camera for expression detection.")
2964
2965     # connect to Pepper (skipped in local mode)
2966     ssh = None
2967     ssh_tts = None
2968     energy_threshold = DEFAULT_ENERGY_THRESHOLD
2969
2970     # Pepper SSH connection: required whenever output goes through the robot
2971     # (TTS, LEDs, gestures, face-tracking). LOCAL_MODE remains the gate for that.
2972     if LOCAL_MODE:
2973         print("\n LOCAL MODE: skipping Pepper connection.")
2974     else:
2975         print(f"\nConnecting to Pepper at {NAO_IP}...")
2976         ssh = ssh_connect()
2977         ssh_tts = ssh_connect()    # dedicated TTS connection
2978         print(" Connected.")
2979
2980     # Pepper stream ~10 fps; detect at 6-7 Hz
2981     if not USE_LOCAL_CAMERA:
2982         print(" Starting Pepper camera stream...")
2983         threading.Thread(target=pepper_video_loop,
2984                         args=(ssh,), daemon=True).start()
2985         threading.Thread(target=pepper_camera_receive_loop,
2986                         args=(NAO_IP,), daemon=True).start()
2987         threading.Thread(target=detect_thread_loop,
2988                         args=(face_model, face_cascade), daemon=True).start()
2989         print(" Pepper camera stream threads started.")
2990
2991     # Mic-side ambient calibration: pick whichever microphone INPUT_IS_LOCAL says.
2992     # In gaze21 hybrid mode this is the Mac mic even though Pepper is connected.
2993     if INPUT_IS_LOCAL:
2994         print("\nCalibrating Mac microphone ambient noise level...")
2995         local_calibrate_ambient()
2996     else:
2997         print("\nCalibrating ambient noise level (stay quiet for 3 seconds)...")
2998         energy_threshold = nao_calibrate_ambient(ssh)
2999
3000     dashboard = GazeDashboard(ssh=ssh, ssh_tts=ssh_tts)
3001     print(" Dashboard launched.")
3002
3003     conv_thread = threading.Thread(
3004         target=conversation_loop,
3005         args=(dashboard, face_model, face_cascade, speech_model,
3006             local_camera, ssh, ssh_tts, energy_threshold),

```

```

3007         daemon=True,
3008     )
3009     conv_thread.start()
3010
3011     # tkinter mainloop on main thread (macOS-required); keep camera preview and GUI responsive
3012     ↳ whilst the conversation loop blocks on I/O
3013     dashboard.root.mainloop()
3014
3015 if __name__ == "__main__":
3016     main()

```